arXiv:2312.11556v4 [cs.CV] 31 May 2025

💫 StarVector: Generating Scalable Vector Graphics Code from Images and Text

Juan A. Rodriguez[1,2,4] Abhay Puri[1] Shubham Agarwal[1,2] Issam H. Laradji[1,5] Pau Rodriguez[1]
Sai Rajeswar[1,2] David Vazquez[1] Christopher Pal[1,2,3] Marco Pedersoli[1,4]

[1]ServiceNow Research [2]Mila - Quebec AI Institute [3]Canada CIFAR AI Chair
[4]ÉTS, Montréal, Canada [5]UBC, Vancouver, Canada
https://starvector.github.io/

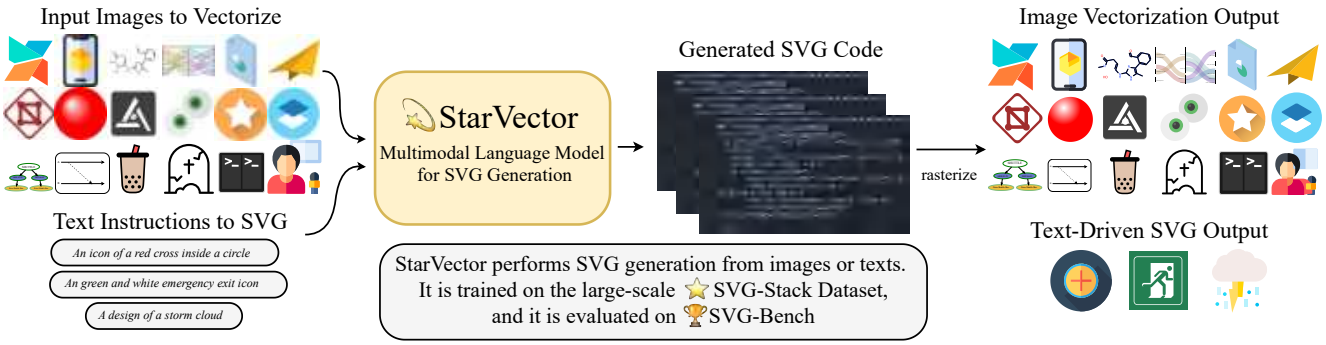


Figure 1. **StarVector: A foundation model for SVG generation.** StarVector's multimodal architecture allows input from raster images or text instructions. It converts a variety of raster visuals, including icons, logos, and technical diagrams, into vector graphics or generates new SVGs from text. (Left) Inputs: raster images and text. (Right) Outputs: vectorized images (SVG)

Abstract

Scalable Vector Graphics (SVGs) are vital for modern image rendering due to their scalability and versatility. Previous SVG generation methods have focused on curve-based vectorization, lacking semantic understanding, often producing artifacts, and struggling with SVG primitives beyond path curves. To address these issues, we introduce StarVector, a multimodal large language model for SVG generation. It performs image vectorization by understanding image semantics and using SVG primitives for compact, precise outputs. Unlike traditional methods, StarVector works directly in the SVG code space, leveraging visual understanding to apply accurate SVG primitives. To train StarVector, we create SVG-Stack, a diverse dataset of 2M samples that enables generalization across vectorization tasks and precise use of primitives like ellipses, polygons, and text. We address challenges in SVG evaluation, showing that pixel-based metrics like MSE fail to capture the unique qualities of vector graphics. We introduce SVG-Bench, a benchmark across 10 datasets, and 3 tasks: Image-to-SVG, Text-to-SVG generation, and diagram generation. Using this setup, StarVector achieves state-of-the-art performance, producing more compact and semantically rich SVGs.

1. Introduction

Vector graphics represent an archetypal form of image representation, where visual compositions are constituted by scalable primitive shapes [33, 43, 50, 50]. For modern image rendering, Scalable Vector Graphics (SVGs) [60] have become the standard for representing vector graphics. The SVG format [25] provides a comprehensive set of primitives and styling options. At its core, the *path* represents basic curves [60]. Combined with primitives like *polygon* or *ellipse*, SVGs define complex designs precisely.

The task of *image vectorization*, i.e., converting pixel-based raster images into SVGs, stands as a fundamental challenge in vector graphics. The main challenge lies in developing methods that generalize across diverse domains, from fonts and logos to complex illustrations and diagrams [7, 8, 69, 70]. Traditional approaches often rely on approximating images through multiple *paths* [43, 50, 51, 59, 87]. This strategy can be inefficient as shown in Fig. 2 (Right). For instance, a circle shape could be represented as long *path* or, more precisely and compactly, as a single `<circle/>` primitive. Similarly, text elements should be vectorized as editable `<text/>` primitives to retain the original textual content. This balance between curve-based shape approximation and accurately recognizing primitives

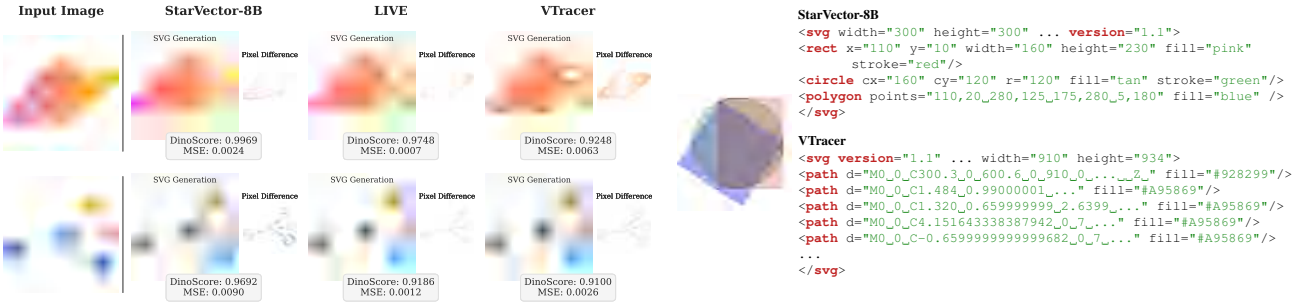


Figure 2. **(Left) Image Vectorization** results using StarVector-8B, LIVE, and VTracer. Each row shows the input image, generated SVGs, and pixel-wise difference maps to highlight accuracy. StarVector-8B better preserves shapes, color gradients, and text, despite minor misplacements. Notably, *MSE often misaligns with visual quality*, e.g., regarding the 'planet' example, StarVector's MSE (0.009) is higher than LIVE's (0.0012) and VTracer's (0.0039), yet StarVector preserves the color gradient. For the 'diagram' example, StarVector preserves the text. DinoScore better reflects these details, consistently favoring StarVector. **(Right) Curve vs Primitive-based Vectorization.** SVG code generated by StarVector and VTracer for the given image. StarVector effectively leverages shape primitives, resulting in a compact vectorization. VTracer decomposes the image into numerous paths, resulting in a more complex result with less semantic clarity.

has been previously unexplored and remains a core challenge in modern vectorization.

Vectorization approaches fall into two main categories: traditional image processing and deep learning (DL)-based methods. Image processing techniques [51, 59, 87] trace vector curves via pixel-level analysis but often generate overly complex representations with artifacts and lack semantic understanding (Figure 2). DL methods [13, 14, 67, 89] improve vector modeling using latent variable models and differentiable rendering [43, 50] but struggle with generalization and underutilize SVG primitives. This limits their effectiveness for complex SVGs like scientific diagrams and hinders their application in modern multimodal tasks such as text-driven SVG generation [24, 65, 72, 96].

Recent advancements in Multimodal Large Language Models (MLLMs) [2, 45] have integrated visual understanding into transformer [84] architectures while demonstrating strong code generation capabilities [1, 42, 49, 55]. Building on these developments, *we introduce image vectorization as an inverse rendering and code generation task*, leveraging MLLMs to generate SVG code directly from input images. This approach naturally encompasses the full range of SVG primitives, enhancing both semantic understanding and generation capabilities (Table 6).

We introduce 💫 StarVector, a foundational MLLM for SVG generation. StarVector processes both images and text instructions to produce compilable SVG code, leveraging SVG primitives to accurately represent vector graphics. We build upon the **Star**Coder works [42, 49] to connect the code generation research with SVG generation. Figure 3 describes the model architecture. It integrates an image encoder that projects images into visual tokens, and a transformer language model for learning the relationships between instructions, visual features, and SVG code sequences, to perform image vectorization (Image-to-SVG) or text-driven SVG Generation (Text-to-SVG) tasks. StarVector performs primitive-aware vectorization through learned semantic understanding, effectively leveraging SVG primitives without explicit pixel reconstruction objectives. To address the lack of large-scale SVG datasets for training StarVector, we introduce SVG-Stack, with over 2M SVG samples paired with rendered images and text descriptions.

Additionally, we find that conventional metrics like MSE fail to adequately assess vector graphics quality, as demonstrated in Figure 2. Instead, we propose DinoScore, a perceptual similarity metric that better correlates with visual human perception, and introduce SVG-Bench, an evaluation framework spanning 10 datasets and 3 tasks: Image-to-SVG, Text-to-SVG, and diagram generation.

Contributions

1. We introduce 💫 **StarVector**, an MLLM capable of image vectorization and text-driven SVG Generation, uniquely preserving SVG primitives rather than producing multiple curves—a previously unexplored skill.
2. We create ⭐ **SVG-Stack**, a large-scale dataset with 2M samples, supporting Image-to-SVG and Text-to-SVG.
3. We develop 🏆 **SVG-Bench**, an MLLM benchmark with 10 datasets across 3 SVG tasks.
4. We conduct extensive experiments and evaluations, including human assessments, demonstrating StarVector's advantages in primitive recognition and highlighting the limitations of pixel-wise metrics for SVG evaluation.

2. Related Work

SVG Generation Methods. Early image vectorization efforts [92] primarily relied on traditional techniques [21, 44, 95] such as segmentation and polynomial curve fitting [43, 51, 59, 87]. With advancements in deep learn-

ing, new approaches have emerged. For instance, SVG-VAE [47] employs a class-conditional Variational Autoencoder (VAE) [34] to predict a latent style vector and generate SVGs using an LSTM decoder [30]. DeepSVG [14] introduces a hierarchical VAE architecture using transformers for SVG path representation, while Im2Vec [67] converts pixel images into latent representations that can be decoded into paths with a recurrent neural network. However, these methods are limited to basic primitives like paths, resulting in a performance gap compared to traditional image processing methods. Concurrently, BeyondPixels [99] utilizes LLMs to predict command sequences in synthetic SVGs. These trends highlight the potential of LLM models for improved generalization in SVG generation.

Recent results in image generation using diffusion [29, 73] or autoregressive [23, 64, 96] models and the success of text-to-image generation [6, 65, 72] have inspired the research into text-conditioned SVG generation. VectorFusion [33] leverages a strong text-to-image diffusion model to find the SVG via iterative optimization. Some work has focused on SVG editing from textual inputs [12]. CLIPasso [86] uses a CLIP distance loss to iteratively refine SVG from sketches. These solutions are slow due to their iterative nature. IconShop [94] trains a BERT [20] model for text-conditioned SVG generation of icons, but their method is restricted to using paths.

In contrast, we train an MLLM on the inverse rendering image vectorization task, leveraging visual understanding to produce accurate SVG code with optimal use of primitives.

SVG Benchmarks and Datasets. Previous work on SVG datasets and benchmarks has been limited. Existing Image-to-SVG datasets mainly focus on fonts, icons, and emojis [12–14, 67, 89, 94], offering limited variety for broader SVG types and primitives. Recent text-driven SVG generation efforts [12, 16, 94] leverage large language models [56] for synthetic captions, but limited dataset accessibility hampers reproducibility. Evaluation methods also face challenges. Existing pixel-based metrics like MSE [14, 47, 67] fail to capture the fidelity and structure of SVGs, overlooking aspects like line definition and primitive usage (Figure 2, illustrates how MSE can be misleading). Recent benchmarking efforts [54, 100] focus on caption-based generation or editing but remain limited in scope.

To address these gaps, we introduce SVG-Stack, a large-scale dataset for diverse SVG generation tasks from both images and text, and SVG-Bench, a unified benchmark with datasets, tasks, and metrics tailored for SVGs. Together, they provide a solid foundation for improving SVG model training and evaluation.

Multimodal Language Models. Large language models (LLMs) have achieved great success in natural language processing (NLP) [10, 55, 61, 82–84], especially in code generation tasks [9, 15, 17, 26, 32, 35, 42, 53]. Recent trends in Multimodal Large Language Modeling (MLLM) [2, 45, 56, 88] have allowed infusing image understanding into text-only models, for tasks like visual-question answering (VQA) [4] or image captioning [39–41]. Current approaches define an image encoder for computing visual tokens from images, that can be processed by an LLM [2, 45, 56, 88]. Multiple works use a Vision Transformer [22] backbone with pre-trained weights like CLIP [62]. We get inspiration from these architectures for training an MLLM on the tasks of image vectorization and text-driven SVG Generation (also referred to as Image-to-SVG and Text-to-SVG).

3. ⭐ SVG-Stack Dataset

To address the lack of large-scale SVG datasets for training foundational SVG models, we introduce SVG-Stack, a dataset containing 2.1 million SVG samples for training, 108k for validation, and 5.7k for testing. Each SVG is paired with its corresponding raster image and descriptive text, making the dataset ideal for multi-modal learning. SVG-Stack is sourced from The Stack [35], a diverse collection of code samples from various software languages. Our selection builds upon the initial filtering and de-duplication processes conducted in [3, 35, 42]. We perform additional filtering to ensure non-duplicate SVG samples.

These samples come from publicly available repositories on GitHub, providing a rich variety of real-world SVGs used across websites, graphic designs, and more. A key advantage of our approach is the inclusion of SVGs with diverse syntactic structures, varying templating approaches, different header formats, framework-specific implementations, and full support for SVG primitives. This diversity significantly enhances model generalization capabilities. This approach contrasts with previous datasets that primarily focused on a narrow subset of SVG types [14, 47, 94]. SVG-Stack represents the first large-scale pre-training dataset for SVG generation, with permissively licensed samples [35], bringing together a broad spectrum of SVGs that closely mirror the diversity seen across the Web. Table 5 and Figures [7, 8, 6] in Appendix 8 provide a comparison of available SVG datasets, showcasing the breadth of test examples found in SVG-Stack.

Data Processing and Curation. Our data processing begins with extracting SVG samples from TheStack [35], followed by a comprehensive deduplication process based on filenames, SVG code, and metadata. We utilized CairoSVG [36] for rasterization, removing samples that produced completely white images. To optimize sequence length and improve visualization, we eliminated comments and XML headers from the SVG code.

Synthetic Generation of Text Instructions. To enable

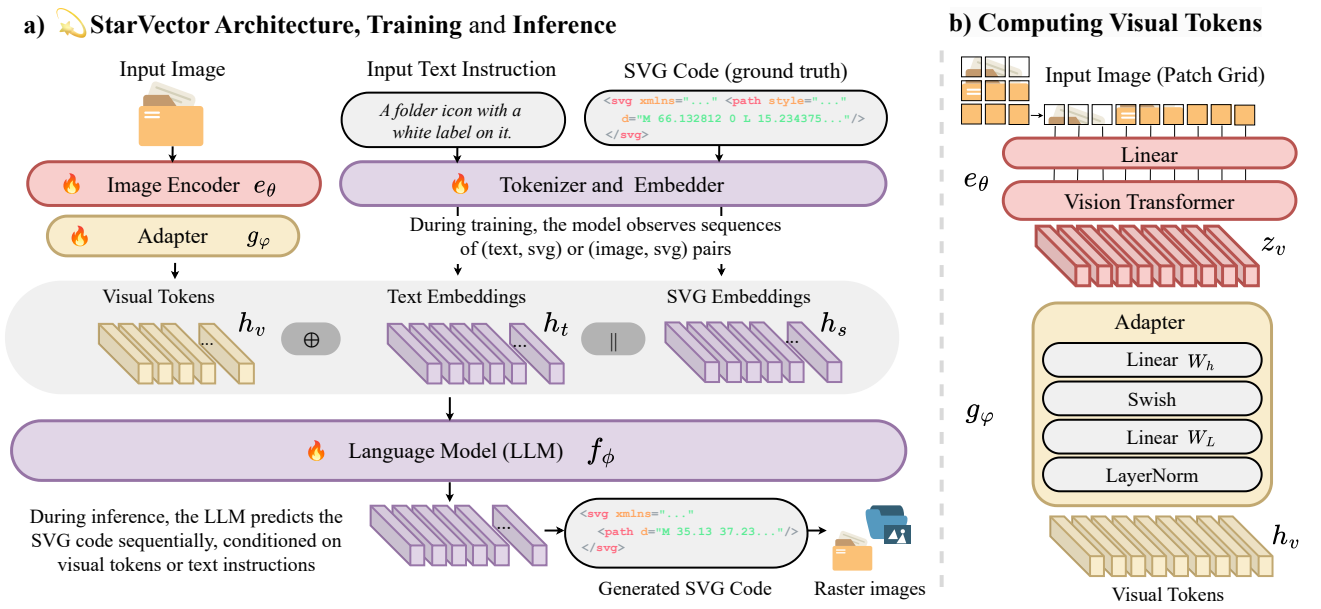


Figure 3. **a) StarVector Architecture**. Images are projected into visual tokens via an Image Encoder and Adapter, aligned with the Language Model's hidden space. Text conditioning uses the LLM's tokenizer and embedder. $\oplus$ denotes mutually exclusive addition of image or text features to SVG tokens, while $\|$ indicates sequence concatenation. During training, the model maps token sequences (visual or textual) to SVG code, and at inference, SVG code is generated sequentially. **b) Visual Token Computation**. We use a Vision Transformer (ViT) to process image patches into hidden features, which are projected through a non-linear Adapter to form visual tokens.

Text-to-SVG generation, we augment our 2M samples with synthetically generated textual captions describing the rasterized images. We employ open-source image captioning models, specifically BLIP2 [41] and Llava [45], resulting in a comprehensive dataset of 4 million paired textual captions and SVGs. Detailed information about the annotation process is available in Appendix 8.4.

Data Augmentation. We implement several SVG-specific augmentation techniques to enhance model robustness. Rather than storing static-resolution raster images, we perform rasterization during data collation, enabling dynamic SVG transformations without significant training overhead. Our augmentation pipeline includes modifications to image resolution, rotation, translation, scaling, and color properties. We leverage open-source libraries svgpathtools [74] and bs4 [68] for SVG manipulation. Ablation studies demonstrate that these augmentation techniques significantly improve model performance.

4. StarVector

4.1. Architecture

StarVector is a foundational MLLM for SVG generation, trained for Image-to-SVG and Text-to-SVG Generation. It effectively uses image semantics to identify and utilize shape primitives, producing precise and compact SVG outputs. This capability emerges as the model learns to predict SVG code directly, while it is trained on a large SVG dataset (SVG-Stack). *We frame the task of image vectorization as an inverse rendering and code generation problem*, where images are represented as visual tokens that precede the sequence of SVG code tokens. During generation, an image is converted into visual tokens, prompting StarVector to predict the SVG code following a vectorization trigger token. As depicted in Figure 3, our architecture employs a large language model (LLM) f_ϕ and an image encoder e_θ, parameterized by trainable parameters ϕ and θ, respectively.

Visual Tokens. For each input image x_v, the Image Encoder provides flattened grid features $z_v = e_\theta(x_v)$. A Vision Transformer (ViT) [62] is utilized to define $e_\theta(\cdot)$. All features from the last transformer layer are used, as we require high visual expressivity. LLM Adapter g_φ is devoted to projecting the visual features z_v into the dimensionality of the LLM, creating visual tokens h_v:

$$h_v = g_\varphi(z_v), \text{where } z_v = e_\theta(x_v) \tag{1}$$

As depicted in Figure 3-b, the LLM Adapter g_φ performs a non-linear projection of the image embeddings into the LLM embedding space, producing a set of visual token embeddings (or visual tokens). This transformation aligns the image representations with the LLM, effectively bridging the visual and SVG code modalities. The Adapter is

composed of a sequence of fully connected (FC) layers with Swish [63] activation function and Layer Normalization [38]. We initialize the adapter parameters φ using the normal distribution. We initialize the image encoder parameters θ using the public weights of CLIP ViT-L/14 [62].

$$g_\varphi(z_v) = \text{LayerNorm}(W_L \cdot \text{Swish}(W_h \cdot z_v)) \quad (2)$$

Language Modeling. For each image x_v, we define x_t as its corresponding textual caption and x_s as the SVG code. For each sample, we have a tuple (x_v, x_t, x_s). Training sequences are constructed by concatenation: (x_v, x_s) for Image-to-SVG tasks and (x_t, x_s) for Text-to-SVG tasks. To simplify, we use x_c to represent the conditioning sequence, which is either x_v or x_t, depending on the task.

As depicted in Figure 3, both textual x_t and SVG x_s sequences are processed by a tokenizer and an embedder, which converts text strings to tokens and tokens to embeddings. This embedding operation has trainable parameters. We model the conditional probability of SVG sequences as:

$$p(x_s \mid x_c) = \prod_{i=1}^{L} p(x_{s,i} \mid x_{s,<i}, x_c), \quad (3)$$

where L is the length of the SVG sequence x_s. This formulation allows us to use a generative objective with next-token cross-entropy over the SVG sequence. During inference, only the conditioning sequence x_c is given as input, and the SVG code is sampled autoregressively until the ending `<svg-end>` token is reached.

StarVector Variants. We define two variants of StarVector to explore its scaling behavior, varying in image resolution, LLM parameter count, and context length.

1. **StarVector-1B** is initialized with a CLIP ViT-B/32 [62] image encoder, processing images at a 224×224 resolution to produce 257 visual tokens. The LLM uses StarCoder-1B [42] with a context length of 8192 tokens.
2. **StarVector-8B** employs a SigLip (siglip-so400m-patch14-384) [97] image encoder, processing images at a 384×384 resolution to yield 576 visual tokens. This model utilizes StarCoder2-7B, offering an expanded context length of 16k tokens. With a total of 8B parameters, we investigate how scaling can yield more precise and compact SVGs due to higher image resolution and enhanced LLM capacity.

Inference. During generation, StarVector processes an input image *or* text, converting it into tokens, and then predicts subsequent tokens auto-regressively to produce SVG code. This code is rasterized with CairoSVG [36], generating an image. A key challenge during generation is ensuring both 1) syntactically valid SVGs and 2) SVGs optimized for compactness and precision. Decoding introduces stochasticity, and limited context length can result in incomplete SVGs. We find that StarVector is sensible to temperature, length penalty, and logit bias, i.e., adding more weight to certain tokens, like the *<svg-end>* token which encourages valid SVG outputs (properly closed SVG). We introduce them as inference hyperparameters. To further improve quality, we generate k samples with varied parameters, ranking them based on DinoScore.

5. 🏆 SVG-Bench: Evaluation Suite for SVG

In response to the limited benchmarks for SVG evaluation and to unify evaluation practices [14, 47, 67], we introduce SVG-Bench. This benchmark assesses SVG methods across Image-to-SVG, Text-to-SVG, and Diagram Generation tasks, encompassing various SVG types from simple graphics like icons and fonts to complex diagrams with multiple primitives. Dataset statistics are presented in Table 5.

5.1. Tasks and Benchmarks

SVG-Bench focuses on the following tasks. For details on dataset curation and visual examples, see Appendix 8.

1. **Image-to-SVG**: This task evaluates converting images to SVGs across varying complexities. We introduce *SVG-Fonts*, *SVG-Emoji*, *SVG-Icons*, and *SVG-Stack*, increasing in complexity. While prior works used similar datasets [14, 47, 67], access has often been unclear. We provide standardized train, validation, and test splits, along with simplified versions containing only *paths* for compatibility with certain methods.
2. **Text-to-SVG**: We evaluate the model's ability to generate SVGs from text instructions. This includes the *SVG-Stack* test set, which provides two textual descriptions per image, and the SVG-FIGR dataset, which is sourced from FIGR-8-SVG [16, 94] dataset, enabling the generation of simpler (path-only) icons from text.
3. **Diagram Generation**: We assess the model's performance in generating diagrams, a specific type of SVG that involves text, rectangles, and arrow primitives. For this, we create the *SVG-Diagrams* test set by extracting samples from SVG-Stack, including textual captions.

Evaluation Metrics. To compute benchmark scores, we define the following metrics: For image vectorization tasks, we use Mean Squared Error (MSE), Structural Similarity Index (SSIM) [90, 91], and Learned Perceptual Image Patch Similarity (LPIPS) [98]. To address the limitations of pixel-based metrics (see Figure 2), we propose DinoScore [57], which computes L2 distance between DinoV2 features. Token Length (Tokens) measures the size of the SVG samples. We use the StarCoder [42] tokenizer to tokenize SVG code and compute the average length. These metrics are also used for Diagram Generation. For Text-to-SVG, we build

Table 1. **Image Vectorization Results.** Image processing methods (denoted by †) excel in pixel-based metrics (SSIM and MSE) while StarVector models lead in semantic-based metrics (DinoScore and LPIPS). StarVector shows better performance in SVG-Stack, SVG-Fonts, and SVG-Icons but underperforms in SVG-Emoji due to limited training data. We highlight the token lengths of generated SVGs from different models, comparing them to the average token count in test examples (shown in gray below the "Tokens" header). Token counts close to the actual number are marked in green, while the largest counts are highlighted in red. *Notably, methods that perform well on MSE tend to utilize a large number of tokens*, whereas StarVector shows remarkable compression.

	SVG-Stack					SVG-Fonts					SVG-Icons					SVG-Emoji				
Method	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Tokens** 2,822	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Tokens** 3,536	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Tokens** 3,085	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Tokens** 5,618
AutoTrace†	0.942	0.063	0.930	0.009	**59.1k**	0.954	0.025	0.968	0.006	**30.8k**	0.946	0.053	0.937	0.014	**56.7k**	0.975	0.077	0.902	0.011	**94.0k**
Potrace†	0.898	0.139	0.856	0.036	7.5k	0.967	**0.009**	**0.988**	0.002	4.2k	0.972	**0.023**	0.973	**0.004**	12.0k	0.882	0.267	0.780	0.067	9.7k
VTracer†	0.954	0.062	0.883	0.010	9.7k	0.964	0.027	0.888	0.009	4.5k	0.940	0.062	0.914	0.017	20.0k	**0.981**	0.074	0.894	0.008	15.7k
Im2Vec	0.692	0.291	0.765	0.181	**4.3k**	0.733	0.140	0.837	0.135	**4.3k**	0.754	0.150	0.889	0.055	**4.3k**	0.732	0.465	0.774	0.126	**3.8k**
LIVE	0.934	0.059	**0.953**	**0.003**	18.3k	0.956	0.013	0.977	**0.001**	18.3k	0.959	0.035	0.973	**0.004**	18.2k	0.969	**0.060**	**0.958**	**0.002**	18.3k
DiffVG	0.810	0.156	0.856	0.019	19.7k	0.821	0.051	0.959	0.007	19.7k	0.952	0.056	0.956	0.015	19.8k	0.814	0.242	0.776	0.034	19.7k
GPT-4-V	0.852	0.317	0.711	0.195	443	0.842	0.198	0.749	0.197	279	0.848	0.238	0.755	0.144	524	0.850	0.344	0.712	0.170	672
StarVector-1B	0.926	0.149	0.840	0.078	**3.7k**	0.978	0.022	0.961	0.022	**2.4k**	0.975	0.040	0.931	0.026	**3.5k**	0.929	0.217	0.820	0.063	**4.8k**
StarVector-8B	**0.966**	**0.058**	0.947	0.026	**5.3k**	**0.982**	0.030	0.946	0.029	**3.0k**	**0.984**	0.035	**0.975**	0.012	**2.8k**	0.943	0.193	0.829	0.052	**6.7k**
	SVG-Stack$_{sim}$					**SVG-Fonts$_{sim}$**					**SVG-Icons$_{sim}$**					**SVG-Emoji$_{sim}$**				
AutoTrace†	0.945	0.063	0.922	0.018	**74.1k**	0.928	0.125	0.886	0.050	1.5k	0.915	0.111	0.901	0.044	1.3k	0.963	0.090	0.874	0.029	**134.8k**
Potrace†	0.970	**0.022**	0.968	0.006	12.2k	0.991	0.012	0.983	0.003	7.7k	0.983	0.025	0.976	0.004	10.4k	**0.992**	**0.037**	**0.951**	**0.008**	26.7k
VTracer†	0.935	0.061	0.914	0.020	16.0k	0.946	0.040	0.939	0.013	12.7k	0.945	0.043	0.946	0.012	11.9k	0.948	0.063	0.911	0.021	16.2k
Im2Vec	0.725	0.186	0.892	0.046	**4.3k**	0.857	0.184	0.833	0.096	284	0.860	0.207	0.792	0.129	453	0.695	0.179	0.898	0.045	**3.7k**
LIVE	0.963	0.039	**0.974**	**0.005**	18.3k	0.975	0.016	**0.991**	**0.001**	**18.3k**	0.961	0.030	**0.978**	**0.003**	**18.2k**	0.958	0.075	0.934	0.014	18.2k
DeepSVG	0.907	0.192	0.835	0.071	1.5k	0.928	0.125	0.886	0.050	1.5k	0.915	0.111	0.901	0.044	1.3k	0.822	0.209	0.841	0.074	1.8k
GPT-4 V	0.874	0.226	0.768	0.137	329	0.946	0.040	0.939	0.013	12.7k	0.945	0.043	0.946	0.012	11.9k	0.852	0.212	0.802	0.105	424
StarVector-1B	0.954	0.089	0.870	0.053	**2.9k**	-	-	-	-	-	-	-	-	-	-	0.977	0.073	0.897	0.043	**3.0k**
StarVector-8B	**0.977**	0.074	0.888	0.045	**2.1k**	**0.993**	**0.012**	0.970	0.009	**1.3k**	**0.990**	**0.024**	0.947	0.017	**2.7k**	0.903	0.163	0.791	0.091	**3.2k**

Table 2. **Image Vectorization on SVG-Diagrams**. StarVector outperforms LIVE in DinoScore, LPIPS, and SSIM, while LIVE ranks best in MSE. However, visual results (Fig. 2) confirm that StarVector is the only effective method for SVG generation, underscoring the misalignment of MSE. Additionally, it remains competitive in terms of token length.

	SVG-Diagrams				
Method	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Tokens ↓**
Autotrace†	0.874	0.114	0.883	0.013	90.6k
Potrace†	0.875	0.153	0.862	0.026	22.6k
VTracer†	0.882	0.116	0.877	0.011	15.8k
LIVE	0.870	0.121	0.859	**0.010**	18.3k
DiffVG	0.822	0.170	0.859	0.019	19.8k
StarVector-1B	0.943	0.107	0.862	0.032	9.5k
StarVector-8B	**0.959**	**0.093**	**0.890**	0.027	-

on text-to-image literature [62, 66, 73, 81] and prior Text-to-SVG methods [12, 94], using FID [81], FID-CLIP [94], and CLIP Score [62] to measure image-text alignment.

6. Experiments and Results

We train StarVector (1B and 8B versions) on the *inverse rendering* vectorization task using SVG-Stack dataset. We then fine-tune on the other datasets mentioned in Section 5, as well as for Text-to-SVG task. We evaluate StarVector and other methods on SVG-Bench, focusing on quantitative and qualitative performance, SVG primitive use, and compact-

Table 3. **Usage of *Paths* and Inference Time.** We ablate the use of the *path* primitive across models. LIVE and DiffVG allow setting the number of paths, while VTracer, Autotrace, and StarVector dynamically determine them. More paths generally improve performance. LIVE achieves the best pixel metrics, but StarVector excels in DinoScore. We also report *average inference time per sample*, noting that LIVE is significantly slower. Results are averaged across SVG-Bench datasets.

Method	**# Paths**	**Dino ↑**	**LPIPS ↓**	**SSIM ↑**	**MSE ↓**	**Time (s) ↓**
LIVE	5	0.898	0.137	0.881	0.013	190
	16	0.930	0.064	0.937	0.006	290
	32	0.937	0.057	0.944	0.004	650
	60	0.939	**0.053**	**0.947**	**0.003**	1,412
DiffVG	15	0.781	0.205	0.819	0.066	21
	60	0.844	0.135	0.881	0.018	31
	120	0.895	0.107	0.907	0.013	45
Vtracer	18	0.942	0.067	0.892	0.011	0.09
Potrace	-	0.937	0.109	0.897	0.024	10
AutoTrace	3k	0.951	0.065	0.924	0.010	1
StarVector-1B	8	0.952	0.107	0.883	0.044	41
StarVector-8B	10	**0.963**	0.085	0.911	0.031	74

Table 4. **Results on Text-to-SVG**: We report FID, FID-CLIP (FID-C), and CLIP Score (CLIP) on SVG-Stack and SVG-FIGR. StarVector models outperform all previous baselines in all metrics. We observe improvement when scaling StarVector from 1B to 8B. Results for DeepSVG+GAN [14, 28] and Bert on SVG-FIGR are extracted from [94], while StarVector is trained on the same data and splits. Missing scores are due to limited model access.

	SVG-FIGR			SVG-Stack		
Method	**FID** ↓	**FID-C** ↓	**CLIP** ↑	**FID** ↓	**FID-C** ↓	**CLIP** ↑
DeepSVG+GAN	-	12.011	21.783	-	-	-
Bert	-	35.104	22.035	-	-	-
IconShop	-	4.657	25.746	-	-	-
GPT-4	32.953	19.026	26.088	37.381	9.664	26.228
CodeLlama	29.002	22.536	26.227	34.777	11.152	25.532
StarVector-1B	15.263	3.834	26.342	28.374	6.482	29.372
StarVector-8B	**10.067**	**1.308**	**27.366**	**25.828**	**4.645**	**31.307**

ness. The following sections present the experimental setup and results. Ablation studies on the architecture, data augmentation, and generation can be found in Appendix 10.1.

Baselines. Baselines. For our comparisons, we consider the following model baselines: For Image-to-SVG, we evaluate top image processing algorithms such as Potrace [59], Vtracer [87], and Autotrace [51]. We also report on deep learning methods including DeepSVG (<5M parameters) [14], Im2Vec (<5M) [67], and MLLMs like GPT-4 Vision (>100B) [56]. For Text-to-SVG, we consider methods like IconShop (>1B), DeepSVG+GAN (<5M), and BERT (>1B), as outlined in [94]. Additionally, we evaluate LLMs such as CodeLlama-70b [82] and GPT-4 (>100B) [55]. For more details, see Appendix 9, and Table 6 summarizes the SVG capabilities of all methods.

Training and Inference. We train StarVector-1B with a batch size of 128 and StarVector-8B with 512, using a learning rate of 1e-5 and the AdamW optimizer. StarVector-1B took 7 days on 8 A100 GPUs, while StarVector-8B took 10 days on 64 H100 GPUs, both completing 2 epochs. Full training details are in Appendix 11.2. During inference, we generate $k = 5$ SVG outputs with temperatures ranging from 0 to 1, selecting the one with the highest DinoScore. We set a logit bias of 10 for the <*svg-end*> token and apply top-p nucleus sampling (0.9) with a length penalty of -0.5, using beam search with a size of 1. We utilize vLLM [37] as the backend to accelerate the generation process.

Experimental Setup. StarVector models are initialized as described in Section 4 and all their weights are unfrozen for the Image-to-SVG task using SVG-Stack. We then fine-tune these models on the train sets of SVG-Emoji, SVG-Fonts, and SVG-Icons, including their simplified versions. For the Text-to-SVG task, the image encoder is disregarded. Only the LLM is trained on SVG-Stack and FIRG-SVG. We perform a comprehensive evaluation on SVG-Bench, analyzing the performance of baseline models mentioned above, alongside the StarVector models. We reproduce all baselines by training them on the benchmark's training sets, within the limits of their availability.

6.1. Main Results

Image Vectorization. Table 1 presents vectorization scores of models across 8 benchmarks. StarVector outperforms all other models in terms of DinoScore, achieving the highest score on six out of the eight benchmarks, thereby establishing its dominance in this metric. Results for the other metrics—LPIPS, SSIM, and MSE—are more varied. However, LIVE demonstrates superior performance on the SSIM and MSE metrics across all datasets. It is important to note that models that perform well on MSE tend to generate larger SVG files, as indicated by the number of tokens in the SVG code. This creates overly complex vectors with visible artifacts (see Fig. 4). Specifically, LIVE's SVG outputs average around 18k tokens, while VTracer varies between 4.5k and 20k tokens. In contrast, StarVector averages approximately 3k tokens, closely matching the ground truth token count. This efficiency is primarily attributed to StarVector's effective use of SVG primitives, as illustrated in Figure 2 (right).

Figure 4 presents qualitative results of models, on SVG-Stack and SVG-Diagrams. In terms of visual quality, LIVE, VTracer, and AutoTrace produce artifacts, especially when dealing with small details. Potrace offers more sharp results, but it is monochromatic. StarVector-8B produces superior results on shape preservation and definition. More results and qualitative samples can be found in Appendix 10.

Why is pixel-based MSE not well-suited? Our results reveal significant limitations of pixel-based metrics (MSE, SSIM, LPIPS) for SVG quality assessment. While StarVector shows worse MSE scores compared to other methods in Table 2, visual inspection of the results in Figures 4 and 14 (especially on diagrams) demonstrates StarVector's superior quality. This discrepancy is particularly evident in the 'planet' example (Figure 2), where StarVector preserves color gradients and line definition, yet receives poor MSE scores due to minor pixel misalignments. Human evaluation (Fig. 5) shows a preference for StarVector's outputs over other models, contradicting these metrics.

Pixel-based metrics have limitations due to (a) the prevalence of constant background colors (allowing even empty SVGs to score reasonably), (b) sensitivity to small spatial misalignments, and (c) inability to capture non-smooth artifacts at corners. DinoScore proves more reliable and aligns consistently with visual quality, scoring StarVector higher on well-formed samples while penalizing poorly formatted ones, thanks to robust self-supervised training [57].

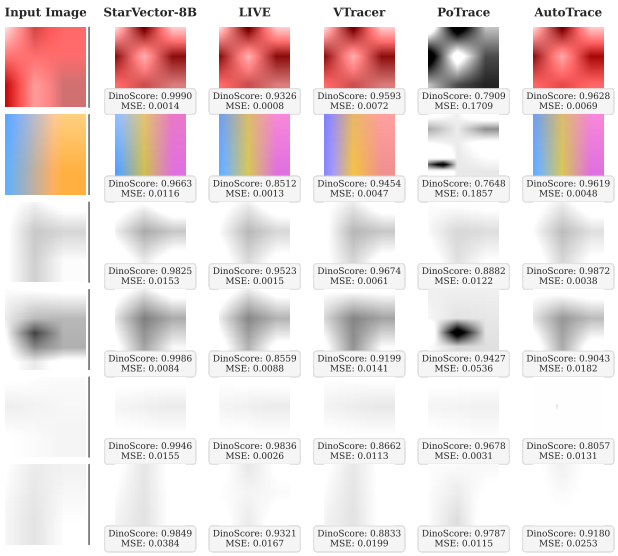


Figure 4. **Image vectorization results** on SVG-Stack (first 3 rows) and SVG-Diagrams (last 3 rows) test sets.

Diagram Generation. Table 2 shows results on SVG-Diagrams, and Figures 4 and 14 provide visual examples. Results highlight that **StarVector is the only method capable of performing diagram generation**, as it uniquely applies the required primitives like rectangles, arrows, and text, whereas other methods produce blobs and curves that attempt to replicate structure and color. Metrics such as DinoScore, LPIPS, capture this advantage, while MSE and SSIM remain poorly aligned. Human evaluations further confirmed the preference for StarVector's outputs.

SVG Primitive Usage. StarVector produces more compact SVGs by optimally using SVG primitives. This innovation combines visual semantic understanding and shape composition with direct SVG code generation, enabling decomposition into basic primitives. As shown in Figure 2 (right), StarVector efficiently represents shapes using primitives, while VTracer relies on a large collection of *paths*. See Appendix 10.4 for more examples (Figure 10) and SVG tag distribution analysis (Figure 30).

Human Evaluation. We conducted human evaluations comparing StarVector-8B with baseline results, involving participants from diverse backgrounds screened for conflicts of interest. Results in Figure 5 show a strong preference for StarVector-8B in all settings, particularly in the SVG-Diagrams tasks, highlighting a disconnect between pixel-based metrics (MSE, SSIM) and visual perception of SVG. While baselines prioritize pixel-perfect reconstruction, humans prefer StarVector's sharp, well-defined shapes and effective use of primitives (Figure 30). Spearman correlations between model metrics (MSE and DINO) and human evaluation show weak correlations for MSE (0.0596 and -0.1002), indicating it is not a strong predictor. In contrast, DinoScore exhibits stronger correlations, with values of -0.6193 and 0.6214, and a strong correlation of 0.7577 between DINO differences and human evaluation differences, highlighting DINO as a more reliable metric.

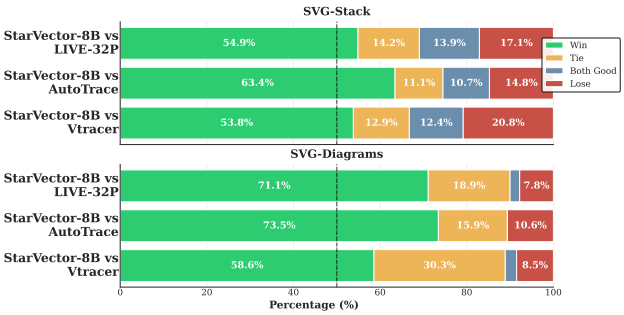


Figure 5. **Human evaluation**. StarVector-8B was evaluated against top-performing baselines: LIVE, AutoTrace, and Vtracer. Results consistently showed a strong human preference for SVG outputs generated by StarVector-8B over the baselines. In total, 1,948 evaluations were collected from 30 unique participants.

Text-to-SVG. Table 4 shows StarVector outperforms baselines on SVG-FIGR and SVG-Stack. While qualitative results show potential, semantic accuracy often suffers due to data quality (see Appendix 10.2).

7. Conclusions

We introduced StarVector, an MLLM that excels in Image-to-SVG and Text-to-SVG generation, delivering superior visual quality, precise line and shape rendering, and optimal SVG primitive usage compared to baselines. To train StarVector, we created SVG-Stack, a diverse dataset that enables generalization across SVG types and primitives. Additionally, we developed SVG-Bench, a unified benchmark with tasks, datasets, and targeted metrics. Our study shows that traditional metrics inadequately capture SVG quality, leading us to propose more effective alternatives.

Limitations and Future Work. StarVector is constrained by its 16k token context, which is inadequate for complex SVGs. It relies primarily on code prediction with minimal visual feedback. Additionally, generation speed is limited by the LLM. Future work may investigate integrating pixel-based signals to overcome these challenges.

Acknowledgements. We sincerely thank Ghazwa Darwiche, Christian Hudon, Fanny Rancourt, and Marie-Ève Marchand for their invaluable administrative and technical support. This work was supported by the Natural Sciences and Engineering Research Council of Canada and Mitacs. Chris Pal acknowledges the Canada CIFAR AI Chair. We also appreciate the human verifiers for their contributions to assessment and data verification.

References

[1] The claude 3 model family: Opus, sonnet, haiku. 2

[2] Jean-Baptiste Alayrac, Jeff Donahue, Pauline Luc, Antoine Miech, Iain Barr, Yana Hasson, Karel Lenc, Arthur Mensch, Katherine Millican, Malcolm Reynolds, et al. Flamingo: a visual language model for few-shot learning. *Advances in Neural Information Processing Systems*, 35: 23716–23736, 2022. 2, 3, 28

[3] Loubna Ben Allal, Raymond Li, Denis Kocetkov, Chenghao Mou, Christopher Akiki, Carlos Munoz Ferrandis, Niklas Muennighoff, Mayank Mishra, Alex Gu, Manan Dey, et al. Santacoder: don't reach for the stars! *arXiv preprint arXiv:2301.03988*, 2023. 3

[4] Stanislaw Antol, Aishwarya Agrawal, Jiasen Lu, Margaret Mitchell, Dhruv Batra, C Lawrence Zitnick, and Devi Parikh. Vqa: Visual question answering. In *Proceedings of the IEEE international conference on computer vision*, pages 2425–2433, 2015. 3

[5] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *arXiv preprint arXiv:1607.06450*, 2016. 40

[6] Yogesh Balaji, Seungjun Nah, Xun Huang, Arash Vahdat, Jiaming Song, Qinsheng Zhang, Karsten Kreis, Miika Aittala, Timo Aila, Samuli Laine, Bryan Catanzaro, Tero Karras, and Ming-Yu Liu. ediff-i: Text-to-image diffusion models with ensemble of expert denoisers. *arXiv preprint arXiv:2211.01324*, 2022. 3

[7] Jonas Belouadi, Anne Lauscher, and Steffen Eger. Automatikz: Text-guided synthesis of scientific vector graphics with tikz. *arXiv preprint arXiv:2310.00367*, 2023. 1, 42

[8] Jonas Belouadi, Simone Paolo Ponzetto, and Steffen Eger. Detikzify: Synthesizing graphics programs for scientific figures and sketches with tikz. *arXiv preprint arXiv:2405.15306*, 2024. 1

[9] Ekaba Bisong and Ekaba Bisong. Google bigquery. *Building Machine Learning and Deep Learning Models on Google Cloud Platform: A Comprehensive Guide for Beginners*, pages 485–517, 2019. 3

[10] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020. 3, 18

[11] Sébastien Bubeck, Varun Chandrasekaran, Ronen Eldan, Johannes Gehrke, Eric Horvitz, Ece Kamar, Peter Lee, Yin Tat Lee, Yuanzhi Li, Scott Lundberg, et al. Sparks of artificial general intelligence: Early experiments with gpt-4. *arXiv preprint arXiv:2303.12712*, 2023. 18, 19

[12] Mu Cai, Zeyi Huang, Yuheng Li, Haohan Wang, and Yong Jae Lee. Leveraging large language models for scalable vector graphics-driven image understanding. *arXiv preprint arXiv:2306.06094*, 2023. 3, 6, 40

[13] Defu Cao, Zhaowen Wang, Jose Echevarria, and Yan Liu. Svgformer: Representation learning for continuous vector graphics using transformers. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 10093–10102, 2023. 2, 13

[14] Alexandre Carlier, Martin Danelljan, Alexandre Alahi, and Radu Timofte. Deepsvg: A hierarchical generative network for vector graphics animation. *Advances in Neural Information Processing Systems*, 33:16351–16361, 2020. 2, 3, 5, 7, 13, 14, 15, 18, 29

[15] Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde de Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021. 3

[16] Louis Clouâtre and Marc Demers. Figr: Few-shot image generation with reptile. *ArXiv*, abs/1901.02199, 2019. 3, 5

[17] Arghavan Moradi Dakhel, Vahid Majdinasab, Amin Nikanjam, Foutse Khomh, Michel C Desmarais, and Zhen Ming Jack Jiang. Github copilot ai pair programmer: Asset or liability? *Journal of Systems and Software*, 203:111734, 2023. 3

[18] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Advances in Neural Information Processing Systems*, 35:16344–16359, 2022. 41

[19] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE conference on computer vision and pattern recognition*, pages 248–255. Ieee, 2009. 40

[20] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. Bert: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint arXiv:1810.04805*, 2018. 3

[21] James Richard Diebel. *Bayesian Image Vectorization: the probabilistic inversion of vector image rasterization*. Stanford University, 2008. 2

[22] Alexey Dosovitskiy, Lucas Beyer, Alexander Kolesnikov, Dirk Weissenborn, Xiaohua Zhai, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer, Georg Heigold, Sylvain Gelly, et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint arXiv:2010.11929*, 2020. 3, 40

[23] Patrick Esser, Robin Rombach, and Bjorn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 12873–12883, 2021. 3, 26, 28, 39, 40

[24] Patrick Esser, Sumith Kulal, Andreas Blattmann, Rahim Entezari, Jonas Müller, Harry Saini, Yam Levi, Dominik Lorenz, Axel Sauer, Frederic Boesel, et al. Scaling rectified flow transformers for high-resolution image synthesis. In *Forty-first International Conference on Machine Learning*, 2024. 2

[25] Jon Ferraiolo, Fujisawa Jun, and Dean Jackson. *Scalable vector graphics (SVG) 1.0 specification*. iuniverse Bloomington, 2000. 1

[26] Leo Gao, Stella Biderman, Sid Black, Laurence Golding, Travis Hoppe, Charles Foster, Jason Phang, Horace He, Anish Thite, Noa Nabeshima, et al. The pile: An 800gb dataset

of diverse text for language modeling. *arXiv preprint arXiv:2101.00027*, 2020. 3

[27] Xavier Glorot and Yoshua Bengio. Understanding the difficulty of training deep feedforward neural networks. In *Proceedings of the thirteenth international conference on artificial intelligence and statistics*, pages 249–256. JMLR Workshop and Conference Proceedings, 2010. 40

[28] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial networks. *Communications of the ACM*, 63(11):139–144, 2020. 7

[29] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 3

[30] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural computation*, 9(8):1735–1780, 1997. 3

[31] Ari Holtzman, Jan Buys, Li Du, Maxwell Forbes, and Yejin Choi. The curious case of neural text degeneration. *arXiv preprint arXiv:1904.09751*, 2019. 27, 41

[32] Hamel Husain, Ho-Hsiang Wu, Tiferet Gazit, Miltiadis Allamanis, and Marc Brockschmidt. Codesearchnet challenge: Evaluating the state of semantic code search. *arXiv preprint arXiv:1909.09436*, 2019. 3

[33] Ajay Jain, Amber Xie, and Pieter Abbeel. Vectorfusion: Text-to-svg by abstracting pixel-based diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1911–1920, 2023. 1, 3

[34] Diederik P Kingma and Max Welling. Auto-encoding variational bayes. *arXiv preprint arXiv:1312.6114*, 2013. 3

[35] Denis Kocetkov, Raymond Li, Loubna Ben Allal, Jia Li, Chenghao Mou, Carlos Muñoz Ferrandis, Yacine Jernite, Margaret Mitchell, Sean Hughes, Thomas Wolf, et al. The stack: 3 tb of permissively licensed source code. *arXiv preprint arXiv:2211.15533*, 2022. 3, 14

[36] Kozea. Cairosvg. https://cairosvg.org/, 2023. 3, 5

[37] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the 29th Symposium on Operating Systems Principles*, pages 611–626, 2023. 7

[38] Jimmy Lei Ba, Jamie Ryan Kiros, and Geoffrey E Hinton. Layer normalization. *ArXiv e-prints*, pages arXiv–1607, 2016. 5

[39] Chenliang Li, Haiyang Xu, Junfeng Tian, Wei Wang, Ming Yan, Bin Bi, Jiabo Ye, Hehong Chen, Guohai Xu, Zheng Cao, et al. mplug: Effective and efficient vision-language learning by cross-modal skip-connections. *arXiv preprint arXiv:2205.12005*, 2022. 3

[40] Junnan Li, Dongxu Li, Caiming Xiong, and Steven Hoi. Blip: Bootstrapping language-image pre-training for unified vision-language understanding and generation. In *International Conference on Machine Learning*, pages 12888–12900. PMLR, 2022.

[41] Junnan Li, Dongxu Li, Silvio Savarese, and Steven Hoi. Blip-2: Bootstrapping language-image pre-training with frozen image encoders and large language models. *arXiv preprint arXiv:2301.12597*, 2023. 3, 4, 14

[42] Raymond Li, Loubna Ben Allal, Yangtian Zi, Niklas Muennighoff, Denis Kocetkov, Chenghao Mou, Marc Marone, Christopher Akiki, Jia Li, Jenny Chim, et al. Starcoder: may the source be with you! *arXiv preprint arXiv:2305.06161*, 2023. 2, 3, 5, 39, 41

[43] Tzu-Mao Li, Michal Lukáč, Michaël Gharbi, and Jonathan Ragan-Kelley. Differentiable vector graphics rasterization for editing and learning. *ACM Transactions on Graphics (TOG)*, 39(6):1–15, 2020. 1, 2, 17, 18, 29

[44] Zicheng Liao, Hugues Hoppe, David Forsyth, and Yizhou Yu. A subdivision-based representation for vector image editing. *IEEE transactions on visualization and computer graphics*, 18(11):1858–1867, 2012. 2

[45] Haotian Liu, Chunyuan Li, Qingyang Wu, and Yong Jae Lee. Visual instruction tuning. *arXiv preprint arXiv:2304.08485*, 2023. 2, 3, 4, 14, 28

[46] Zhuang Liu, Hanzi Mao, Chao-Yuan Wu, Christoph Feichtenhofer, Trevor Darrell, and Saining Xie. A convnet for the 2020s. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11976–11986, 2022. 26, 28, 39, 40

[47] Raphael Gontijo Lopes, David Ha, Douglas Eck, and Jonathon Shlens. A learned representation for scalable vector graphics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 7930–7939, 2019. 3, 5, 13, 14

[48] Ilya Loshchilov and Frank Hutter. Decoupled weight decay regularization. *arXiv preprint arXiv:1711.05101*, 2017. 41

[49] Anton Lozhkov, Raymond Li, Loubna Ben Allal, Federico Cassano, Joel Lamy-Poirier, Nouamane Tazi, Ao Tang, Dmytro Pykhtar, Jiawei Liu, Yuxiang Wei, et al. Starcoder 2 and the stack v2: The next generation. *arXiv preprint arXiv:2402.19173*, 2024. 2, 39

[50] Xu Ma, Yuqian Zhou, Xingqian Xu, Bin Sun, Valerii Filev, Nikita Orlov, Yun Fu, and Humphrey Shi. Towards layer-wise image vectorization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 16314–16323, 2022. 1, 2, 15, 17, 40

[51] Martin Weber. Autotrace. https://github.com/autotrace/autotrace, 2024. 1, 2, 7, 16, 17

[52] Kenton Murray and David Chiang. Correcting length bias in neural machine translation. *arXiv preprint arXiv:1808.10006*, 2018. 41

[53] Erik Nijkamp, Bo Pang, Hiroaki Hayashi, Lifu Tu, Huan Wang, Yingbo Zhou, Silvio Savarese, and Caiming Xiong. Codegen: An open large language model for code with multi-turn program synthesis. *arXiv preprint arXiv:2203.13474*, 2022. 3, 39

[54] Kunato Nishina and Yusuke Matsui. Svgeditbench: A benchmark dataset for quantitative assessment of llm's svg editing capabilities. *arXiv preprint arXiv:2404.13710*, 2024. 3

[55] OpenAI. Gpt-4 technical report, 2023. 2, 3, 7, 19

[56] OpenAI. GPT-4V(ision) System Card. https://cdn.openai.com/papers/GPTV_System_Card.pdf, 2023. Accessed: 2023-11-05. 3, 7, 16, 18, 29

[57] Maxime Oquab, Timothée Darcet, Théo Moutakanni, Huy Vo, Marc Szafraniec, Vasil Khalidov, Pierre Fernandez, Daniel Haziza, Francisco Massa, Alaaeldin El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023. 5, 7

[58] Adam Paszke, Sam Gross, Soumith Chintala, Gregory Chanan, Edward Yang, Zachary DeVito, Zeming Lin, Alban Desmaison, Luca Antiga, and Adam Lerer. Automatic differentiation in PyTorch. In *NeurIPS-W*, 2017. 41

[59] Peter Selinger. Potrace. https://github.com/tatarize/potrace, 2024. 1, 2, 7, 16, 17

[60] Antoine Quint. Scalable vector graphics. *IEEE MultiMedia*, 10(3):99–102, 2003. 1

[61] Alec Radford, Karthik Narasimhan, Tim Salimans, Ilya Sutskever, et al. Improving language understanding by generative pre-training. *openAI*, 2018. 3

[62] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, et al. Learning transferable visual models from natural language supervision. In *International conference on machine learning*, pages 8748–8763. PMLR, 2021. 3, 4, 5, 6, 26, 28, 39, 40

[63] Prajit Ramachandran, Barret Zoph, and Quoc V Le. Searching for activation functions. *arXiv preprint arXiv:1710.05941*, 2017. 5

[64] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation. In *International Conference on Machine Learning*, pages 8821–8831. PMLR, 2021. 3

[65] Aditya Ramesh, Mikhail Pavlov, Gabriel Goh, Scott Gray, Chelsea Voss, Alec Radford, Mark Chen, and Ilya Sutskever. Zero-shot text-to-image generation, 2021. 2, 3

[66] Aditya Ramesh, Prafulla Dhariwal, Alex Nichol, Casey Chu, and Mark Chen. Hierarchical text-conditional image generation with clip latents. *arXiv preprint arXiv:2204.06125*, 1(2):3, 2022. 6

[67] Pradyumna Reddy, Michael Gharbi, Michal Lukac, and Niloy J Mitra. Im2vec: Synthesizing vector graphics without vector supervision. *arXiv preprint arXiv:2102.02798*, 2021. 2, 3, 5, 7, 15, 17, 29

[68] Leonard Richardson. Beautifulsoup. https://www.crummy.com/software/BeautifulSoup/, 2023. 4

[69] Juan A Rodriguez, David Vazquez, Issam Laradji, Marco Pedersoli, and Pau Rodriguez. Figgen: Text to scientific figure generation. *arXiv preprint arXiv:2306.00800*, 2023. 1

[70] Juan A Rodriguez, David Vazquez, Issam Laradji, Marco Pedersoli, and Pau Rodriguez. Ocr-vqgan: Taming text-within-image generation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pages 3689–3698, 2023. 1

[71] Juan A Rodriguez, Nicholas Botzer, David Vazquez, Christopher Pal, Marco Pedersoli, and Issam Laradji. Intentgpt: Few-shot intent discovery with large language models. *arXiv preprint arXiv:2411.10670*, 2024. 19

[72] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models, 2021. 2, 3

[73] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022. 3, 6, 26

[74] Andy S. svgpathtools. https://github.com/mathandy/svgpathtools, 2023. 4

[75] Christoph Schuhmann, Romain Beaumont, Richard Vencu, Cade Gordon, Ross Wightman, Mehdi Cherti, Theo Coombes, Aarush Katta, Clayton Mullis, Mitchell Wortsman, et al. Laion-5b: An open large-scale dataset for training next generation image-text models. *Advances in Neural Information Processing Systems*, 35:25278–25294, 2022. 39, 40

[76] Louis Shao, Stephan Gouws, Denny Britz, Anna Goldie, Brian Strope, and Ray Kurzweil. Generating high-quality and informative conversation responses with sequence-to-sequence models. *arXiv preprint arXiv:1701.03185*, 2017. 41

[77] Noam Shazeer. Fast transformer decoding: One write-head is all you need. *arXiv preprint arXiv:1911.02150*, 2019. 41

[78] Benjamin Spector and Chris Re. Accelerating llm inference with staged speculative decoding. *arXiv preprint arXiv:2308.04623*, 2023. 41

[79] Nitish Srivastava, Geoffrey Hinton, Alex Krizhevsky, Ilya Sutskever, and Ruslan Salakhutdinov. Dropout: a simple way to prevent neural networks from overfitting. *The journal of machine learning research*, 15(1):1929–1958, 2014. 40

[80] Chengjun Tang, Kun Zhang, Chunfang Xing, Yong Ding, and Zengmin Xu. Perlin noise improve adversarial robustness. *arXiv preprint arXiv:2112.13408*, 2021. 14

[81] Lucas Theis, Aäron van den Oord, and Matthias Bethge. A note on the evaluation of generative models. *arXiv preprint arXiv:1511.01844*, 2015. 6

[82] Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023. 3, 7, 19

[83] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288*, 2023. 19

[84] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017. 2, 3

[85] Ashwin K Vijayakumar, Michael Cogswell, Ramprasath R Selvaraju, Qing Sun, Stefan Lee, David Crandall, and

Dhruv Batra. Diverse beam search: Decoding diverse solutions from neural sequence models. *arXiv preprint arXiv:1610.02424*, 2016. 41

[86] Yael Vinker, Ehsan Pajouheshgar, Jessica Y Bo, Roman Christian Bachmann, Amit Haim Bermano, Daniel Cohen-Or, Amir Zamir, and Ariel Shamir. Clipasso: Semantically-aware object sketching. *ACM Transactions on Graphics (TOG)*, 41(4):1–11, 2022. 3

[87] Vision Cortex. VTracer. https://www.visioncortex.org/vtracer-docs, 2023. 1, 2, 7, 16, 17

[88] Peng Wang, Shuai Bai, Sinan Tan, Shijie Wang, Zhihao Fan, Jinze Bai, Keqin Chen, Xuejing Liu, Jialin Wang, Wenbin Ge, et al. Qwen2-vl: Enhancing vision-language model's perception of the world at any resolution. *arXiv preprint arXiv:2409.12191*, 2024. 3

[89] Yizhi Wang and Zhouhui Lian. Deepvecfont: Synthesizing high-quality vector fonts via dual-modality learning. *ACM Transactions on Graphics (TOG)*, 40(6):1–15, 2021. 2, 3

[90] Zhou Wang and Alan C Bovik. Mean squared error: Love it or leave it? a new look at signal fidelity measures. *IEEE signal processing magazine*, 26(1):98–117, 2009. 5

[91] Zhou Wang, Alan C Bovik, Hamid R Sheikh, and Eero P Simoncelli. Image quality assessment: from error visibility to structural similarity. *IEEE transactions on image processing*, 13(4):600–612, 2004. 5

[92] Wikipedia. Comparison of raster-to-vector conversion software — Wikipedia, the free encyclopedia. http://en.wikipedia.org/w/index.php?title=Comparison%20of%20raster-to-vector%20conversion%20software&oldid=1185354750, 2024. [Online; accessed 07-March-2024]. 2

[93] Thomas Wolf, Lysandre Debut, Victor Sanh, Julien Chaumond, Clement Delangue, Anthony Moi, Pierric Cistac, Tim Rault, Rémi Louf, Morgan Funtowicz, et al. Huggingface's transformers: State-of-the-art natural language processing. *arXiv preprint arXiv:1910.03771*, 2019. 41

[94] Ronghuan Wu, Wanchao Su, Kede Ma, and Jing Liao. Iconshop: Text-based vector icon synthesis with autoregressive transformers. *arXiv preprint arXiv:2304.14400*, 2023. 3, 5, 6, 7, 14, 19, 38

[95] Tian Xia, Binbin Liao, and Yizhou Yu. Patch-based image vectorization with automatic curvilinear feature alignment. *ACM Transactions on Graphics (TOG)*, 28(5):1–10, 2009. 2

[96] Jiahui Yu, Yuanzhong Xu, Jing Yu Koh, Thang Luong, Gunjan Baid, Zirui Wang, Vijay Vasudevan, Alexander Ku, Yinfei Yang, Burcu Karagol Ayan, et al. Scaling autoregressive models for content-rich text-to-image generation. *arXiv preprint arXiv:2206.10789*, 2(3):5, 2022. 2, 3

[97] Xiaohua Zhai, Basil Mustafa, Alexander Kolesnikov, and Lucas Beyer. Sigmoid loss for language image pre-training. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 11975–11986, 2023. 5, 40

[98] Richard Zhang, Phillip Isola, Alexei A Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 586–595, 2018. 5

[99] Tong Zhang, Haoyang Liu, Peiyan Zhang, Yuxuan Cheng, and Haohan Wang. Beyond pixels: Exploring human-readable svg generation for simple images with vision language models, 2023. 3

[100] Bocheng Zou, Mu Cai, Jianrui Zhang, and Yong Jae Lee. Vgbench: Evaluating large language models on vector graphics understanding and generation. *arXiv preprint arXiv:2407.10972*, 2024. 3

💫 StarVector: Generating Scalable Vector Graphics Code from Images and Text

Supplementary Material

In the following sections, we provide additional details on the datasets used in this paper, present further experiments, and describe our baselines in detail. We also discuss the StarVector architecture, its training process, and the method for sampling SVG code from the model. Additionally, we provide more insights into SVG-Bench, including the proposed datasets and the different baselines within the evaluation setup. Finally, qualitative results are presented to showcase the strengths and limitations of our foundational model.

8. SVG Datasets in SVG-Bench

Here we describe available SVG datasets in the recent literature. We extend our description of the datasets used for training and evaluating StarVector and other baselines. Earlier SVG datasets proposed in the literature (mainly datasets of emojis and fonts) were not easily accessible due to broken URLs and no direct entry point. Therefore, we provide them as part of SVGBench for easy reproducibility. We introduce splits for train, validation, and testing. The train set is used to optimize the parameter weights of the network. The validation is used for tuning sampling hyperparameters, and the test is used for evaluation. Our model can handle up to 8k context tokens. Therefore, our datasets only consider examples with up to 8,192 tokens. See Table 5 for a complete description of the datasets. See Figures 6, 7, 8 for ground truth examples of the test sets of SVG-Bench.

8.1. Datasets with Simplified SVGs.

We create simplified versions of our four main datasets, i.e. emojis, icons, fonts, and SVG-Stack. This is done because DeepSVG [14] requires a simplification of the SVG in its input. The simplification consists of eliminating complex primitives and using only vector *paths*. Also, color and shapes are abstracted only to use simple line strokes.

8.2. Creating the SVG-Fonts Dataset

To construct the SVG-Font dataset, we replicate the procedure described in SVG-VAE [47][1], which provides a list of public URLs containing open font packages. We download these packages, excluding any with broken URLs. The TTF files are then converted to SFD format, and we further use InkScape[2] to convert them into SVG code. Samples from the test set are shown in Figure 7 (bottom-left). These samples contain only path elements and represent a narrow range of images, each consisting of a single character per image, making them an ideal test case for SVG generation [13, 14, 47].

[1] https://github.com/magenta/magenta/tree/main/magenta/models/svg_vae

[2] https://inkscape.org/

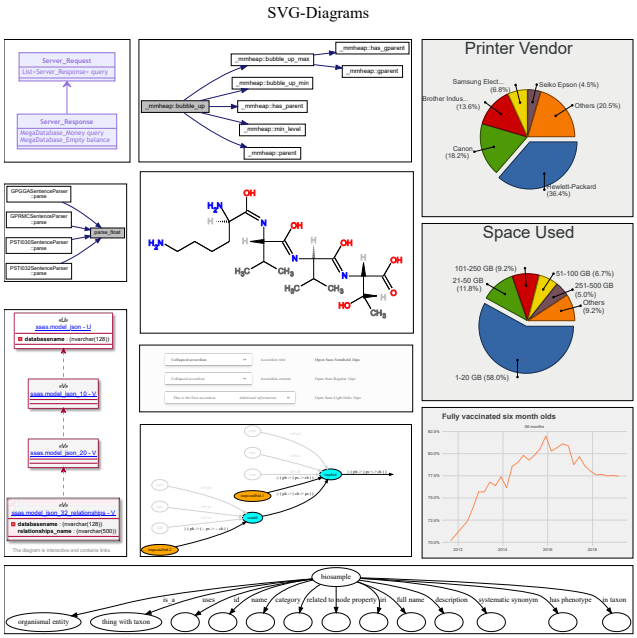


Figure 6. SVG Diagrams examples. These are ground truth SVG examples from the test set. They are presented as SVG, showing the challenge of understanding intricate structures and small texts, with images of variate aspect ratios.

8.3. Creating SVG-Diagrams

We introduce a novel SVG dataset, SVG-Diagrams, which focuses exclusively on diagrams, graphs, workflows, and other designs characterized by discrete elements such as boxes, arrows, and text. To construct this dataset, we filtered all SVGs containing the text element. Table 5 provides detailed statistics about the dataset, and Figure 6 illustrates test samples from SVG-Diagrams.

This new benchmark for diagram generation is highly relevant, as it addresses a use case that cannot be tackled by traditional image-processing models or methods limited to the path primitive. Only approaches capable of leveraging SVG code can fully exploit the use of primitives to generate such structured designs effectively.

8.4. Generating Synthetic Captions on SVG-Stack

To generate textual instructions for vector images, we process SVG-Stack images using visual captioning models.

Table 5. **Summary of datasets.** We offer a summary of statistics about the datasets used in our training and evaluation experiments. This datasets are included in SVG-Bench. The subscript *sim* stands for the simplified version of the dataset, as required by some baselines.

Dataset	Train	Val	Test	Source	Token Length	SVG Primitives	Annotation
SVG-Stack	2,1M	108k	5,7k	TheStack [35]	1,822 ± 1,808	All	Caption
SVG-Stack$_{\text{sim}}$	601k	30,k	1,5k		2k ± 918	Vector path	Caption
SVG-Diagrams	-	-	472		3,486 ± 1,918	All	Caption
SVG-Fonts	1,8M	91,5k	4,8k	Glypazzn [47]	2,121 ± 1,868	Vector path	Font type
SVG-Fonts$_{\text{sim}}$	1,4M	71,7k	3,7k		1,722 ± 723	Vector path	Font type
SVG-Emoji	8,7k	667	668	OpenMoji, NotoEmoji, TweMoji	2,551 ± 1,805	All	Class
SVG-Emoji$_{\text{sim}}$	580	57	96		2,448 ± 1,026	Vector Path	Class
SVG-Icons	80,4k	6,2k	2,4k	DeepSVG [14]	2,449 ± 1,543	Vector path	-
SVG-Icons$_{\text{sim}}$	80,435	2,836	1,277		2,005 ± 824	Vector path	-
SVG-FIGR ‡	270k	27k	3k	IconShop [94]	5,342 ± 2,345	Vector path	Class, Caption

This approach provides a textual description for each image, enabling us to fine-tune our model to follow textual instructions. For this task, we leverage off-the-shelf AI captioners, specifically BLIP2 [41] and Llava [45].

Through prompt engineering, we guide these models in performing the captioning task with reasonable quality. The prompt we used is shared in Prompt 8.4. After automatically captioning all SVG samples in SVG-Stack, we compute the CLIP Score for the text-image pairs generated by the two models—producing two captions per image. Using a CLIP Score threshold of 30, we filter out text captions that fall below this threshold.

Prompt 1. Utilized with BLIP2 and Llava for SVG Captioning: You are a helpful assistant. Your task is to caption the input images with a concise and clear description that represents what are the contents of the image.

8.5. Data Augmentation for SVG

We introduce several data augmentation operations on SVGs that aim to perform minor modifications to the SVG code and rasterize it to get a new sample while training. We include rotation, color, and curve noise. We evaluate this setting on datasets with fewer samples, namely SVG-Emoji and SVG-Icons, as the other two datasets are large enough to do not overfit. Results are shown in Table 8. Both datasets display improvements using these augmentations. We see a substantial uplift for SVG-Emoji, which has limited training data.

We introduce several augmentation operations to SVGs to apply slight changes that help our model learn to generate more precise results — for instance, being able to capture exact colors from the image and encode them in hexadecimal code to insert it in the `fill` attribute of the SVG element. Applying rotations or adding noise to the curve's control points helps the model learn to precisely capture the position of the edges or thickness of the stroke.

We perform random rotations in an angle range. We perform color changes by first parsing the element's color using the `fill` attribute and adding slight white Gaussian noise to the RGB values. We propose curve noise by injecting a small Perlin [80] noise into the control points in Bézier curves. We also experimented with adding Gaussian noise, which resulted in much less natural results. We apply this noise by uniformly sampling a scalar from the interval between 0.01 and 0.05 and use it to scale the noise.

We apply these augmentations directly on the SVG code, which involves parsing the XML code and accessing the attributes and arguments of the primitives defined. We use the libraries `BeautifulSoup`[3] and `SvgPathTools`[4]. Some primitives are simplified using our augmentations.

9. SVG Methods and Baselines

Here, we describe the previous methods and baselines used to compare StarVector's performance in Image-to-SVG and Text-to-SVG generation tasks. We consider previous deep learning-based methods and image-processing methods. We evaluate the baselines with publicly available code in our proposed setup.

9.1. Image-to-SVG Baselines

We reproduce all previous approaches on our proposed SVG-Bench benchmark, as the available results stem from an unclear version of the fonts, emojis, and icons datasets. For theImage-to-SVG task, we consider several baseline

[3] https://www.crummy.com/software/BeautifulSoup/bs4/doc/

[4] https://github.com/mathandy/svgpathtools

Figure 7. **Datasets in SVG-Bench.** Ground truth test examples from the test sets of **SVG-Stack**, **SVG-Emoji**, **SVG-Fonts**, and **SVG-Icons**. We show SVG images.

methods across deep learning and image processing approaches.

In the deep learning methods category, we start with DeepSVG[14], Im2Vec[67], and LIVE[50], using the official implementations with the hyperparameters proposed by the authors, and applying their pre- and post-processing

Figure 8. **Simplified Datasets in SVG Bench.** Ground truth test examples from the simplified test sets of **SVG-Stack**, **SVG-Emoji**, **SVG-Fonts**, and **SVG-Icons**. We show SVG images.

code as required. Additionally, we incorporate the recent GPT-4 Vision[56], which is capable of processing images as input and generating corresponding SVG code as output.

For the image processing-based methods, which do not rely on data-driven learning, we consider VTracer[87], Autotracer[51], and Potrace [59], running them on the test sets of SVG-Bench.

Table 6. **Summary of SVG Methods.** We compare SVG generation methods based on *Image Processing*, *Latent Variable*, *Differentiable Rendering*, and *Multimodal LLM*, evaluating their performance in *SVG Generalization* and *SVG Generation Tasks*. The *SVG Generalization* column shows whether a model generates diverse SVG types (e.g., icons, logos, complex shapes) with ✓ or specializes in a subtype (e.g., emojis, fonts) with ✗. The **SVG Primitive Coverage** column indicates access to all SVG primitives. The table also evaluates **Image Vectorization**, **Text to SVG**, and **Diagram Generation**, using ✓ for support and ✗ for limitations. *DeepSVG requires modifications for image input.

				SVG Coverage and Generalization		*SVG Generation Tasks*		
Method Type	**Model**	**Input**	**Train Supervision**	**SVG Generalization**	**SVG Primitive Coverage**	**Image Vectorization**	**Text to SVG**	**Diagram Generation**
Image Processing	**Vtracer**	Image	Image	✓	✗	✓	✗	✗
	Autotrace	Image	Image	✓	✗	✓	✗	✗
	Potrace	Image	Image	✓	✗	✓	✗	✗
Latent Variable	**Im2Vec**	Image	Image	✗	✗	✓	✗	✗
	DeepSVG	SVG	Vector	✗	✗	✓*	✗	✗
	SVGFormer	SVG	Vector	✗	✗	✓	✗	✗
Diff. Rendering	**DiffVG**	Image	Image	✓	✗	✓	✗	✗
	LIVE	Image	Image	✓	✗	✓	✗	✗
	SAMVG	Image, Text	Image	✓	✗	✓	✗	✗
	SVGDreamer	Image, Text	Image	✓	✗	✗	✓	✗
Multimodal LLM	**GPT-4 V**	Image, Text	SVG	✓	✓	✗	✓	✓
	CodeLlama	Image	SVG	✓	✓	✗	✓	✓
	IconShop	Text	SVG	✗	✗	✗	✓	✗
	StarVector	Image, Text	SVG	✓	✓	✓	✓	✓

Autotrace[5] [51] is a tool designed for converting images to vector graphics, similar to Potrace. It supports various input formats and can output to several vector formats. Autotrace's key feature is its ability to transform pixelated images into smooth, scalable vectors, making it ideal for upgrading images for various applications without losing detail or clarity. Our experiments leverage the Python bindings[6] implementation of AutoTrace.

Potrace[7] [59] is a utility designed to convert images into refined, scalable vector graphics. It accepts input in various bitmap formats and outputs to a selection of vector formats. This functionality is particularly valuable for generating SVG of scanned imagery, such as logos and handwritten documents. We employ a Python library[8], which acts as a wrapper around the original C implementation of Potrace.

VTracer[9] [87] is an image processing algorithm to convert images to SVGs. This 3-step pipeline algorithm relies on the hierarchical clustering of images, which are traced into vectors. First, pixels are converted into paths and then simplified into polygons. In the last step, polygons are smoothened and approximated with a Bezier curve fitter. We use the Python library[10] for experiments, a wrapper over the Rust implementation. Similar to Im2Vec, we scale down all the images to 128X128 resolution. We use all the default values for the image processing engine, which generates a multi-colored SVG.

Im2Vec [67] uses an end-to-end VAE, trained using only image supervision to produce vector graphics. The input rasterized image is encoded to a 'global' latent vector and passed to an RNN to produce latent code for each path. The path decoder decodes these codes into Bezier paths to generate the output SVG. We used the publicly available code[11] to report the results.

We scaled all the images to 128×128 resolution to be compatible with the Im2Vec model. We used a learning rate of 5×10^{-4} and a batch size of 8. We implemented a custom post-processing operation for converting the vector parameters obtained during Im2Vec inference to obtain compilable SVG code.

LIVE, (Layer-wise Image Vectorization) [50] is a method for progressively generating SVGs that closely fit a given raster image by recursively adding and optimizing closed vector *paths*. Using a differentiable renderer (based on DiffVG [43]), LIVE enables direct optimization of paths under raster image supervision while controlling shape complexity by adjusting the number of path segments. It introduces component-wise path initialization, identifying key visual components to ensure efficient topology extraction and minimize redundant shapes. LIVE achieves high-quality reconstructions with fewer paths, reducing SVG file size com-

[5] https://potrace.sourceforge.net/
[6] https://github.com/lemonyte/pyautotrace
[7] https://potrace.sourceforge.net/
[8] https://github.com/tatarize/potrace
[9] https://github.com/visioncortex/vtracer
[10] https://github.com/etjones/vtracer_py
[11] https://github.com/preddy5/Im2Vec

pared to other approaches. Nevertheless, its test time optimization approach makes it time-consuming during generation. We utilized their official open-source implementation [12] with the proposed hyperparameters. This method requires to define a constant number of *paths*; the more paths defined, the more accurate. We have performed an ablation on the number of paths (see Table 3) and found that paths=32 is an optimal value that brings good visual results. However, it takes more than 10 minutes to generate a single SVG, which makes it slow for a professional use case.

DiffVG [43] is a landmark in vector graphics research, pioneering deep learning-based methods with the first differentiable vector graphics rasterization pipeline. By leveraging a combination of anti-aliasing techniques and gradient-based optimization, DiffVG ensures differentiability. Unlike methods relying on non-differentiable curve-to-mesh conversions, DiffVG employs a forward-backward rasterization process, where the forward pass generates anti-aliased images and the backward pass computes gradients with respect to vector graphic parameters. Using the official implementation[13] and proposed hyperparameters, we ablate the number of *paths*, finding *paths*=60 to be optimal. DiffVG balances versatility and performance, achieving approximately 30 seconds per generation while excelling in differentiable rendering tasks.

DeepSVG [14] was introduced as a hierarchical path-based VAE encoder-decoder transformer architecture. Here, input paths are encoded separately using a path encoder and aggregated using a second encoder to produce a latent vector. The decoder uses this latent vector to output the path representations, which provide actual draw commands and arguments. We used the open-source code[14] to reproduce the results on different datasets. However, since the DeepSVG framework only allows simplified SVGs, we report results on the 'simplified' test sets (see Table 1).

This model can only handle simplified SVGs composed of simple line strokes and splines (see examples in Figure 9). Further, it can only process SVGs with eight groups (i.e., groups of shapes or parent nodes) and vector paths of at most 30 commands. To reproduce the DeepSVG baseline, we use the original hyperparameters, including a learning rate of $1e-3$ and a number of epochs of 50. We use a batch size of 200, except for the smaller emoji dataset, where we experiment with a batch size of 50.

[12] https://github.com/ma-xu/LIVE
[13] https://github.com/BachiLi/diffvg
[14] https://github.com/alexandre01/deepsvg

GPT-4 Vision. We use GPT-4V [56] by inserting an image and zero-shot prompting to generate SVG code. Here, we show how one can use prompt engineering [10, 11, 56] to condition the model to generate executable SVG code representing the given image. Prompt 9.2 was used for this endeavor. We use the OpenAI library[15].

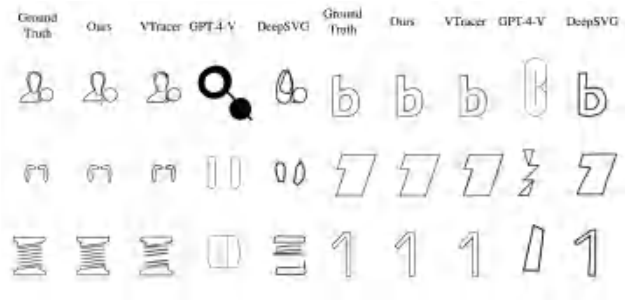


Figure 9. **Image-to-SVG** results on simplified SVG-Icons and SVG-Fonts test set.

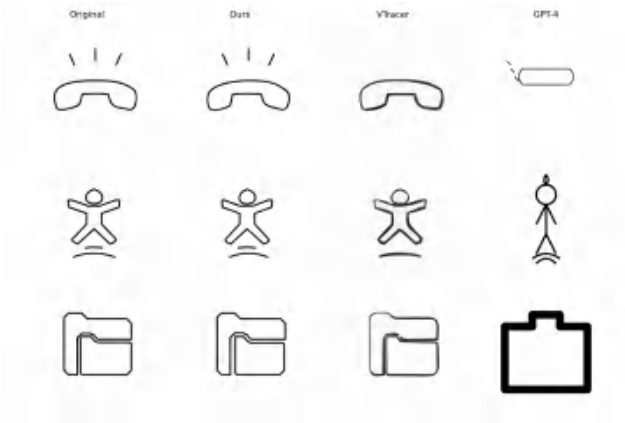


Figure 10. **Image-to-SVG** results on SVG-Icons test set.

Figure 11. **Image-to-SVG** results on SVG-Stack test set. We show cherry-picked failure examples of StarVector.

[15] https://platform.openai.com/docs/libraries

Figure 12. **Image-to-SVG** results on SVG-Emoji test set. We show cherry-picked failure examples of StarVector.

9.2. Text-to-SVG Generation Baselines

For the Text-conditioned SVG generation task, we select baselines based on works that contain reproducible methodologies in public datasets or public code repositories. We reproduce baseline models from their official repositories, respecting the proposed hyperparameters.

CodeLlama, [82, 83] has shown great success in general coding benchmarks. To the best of our knowledge, CodeLlama has seen SVGs during training. Hence, it is reasonable to consider it a strong baseline for text-conditioned SVG generation. We use Anyscale endpoints[16] to generate CodeLlama results.

GPT 4, is a closed source LLM that shows state-of-the-art results in many NLP sceneraios [11, 55, 71]. We evaluate GPT-4's 0-shot ability in generating SVGs when prompted with text inputs. We use OpenAI API[17] to generate results for GPT-4 in the 0-shot setting. Prompt 9.2 was used for the Text-to-SVG task.

Prompt 2. Used on GPT4-V VLM for Image-to-SVG Translation: You are a helpful assistant. Your task is to help researchers write SVG code to reconstruct the provided image as accurately as possible. You should also provide a caption for the image. You are dedicated to solving the task of Image-to-SVG conversion for a robust system. Therefore, you must always respond with the best SVG code you can create. Feel free to use multiple paths to generate a compliant SVG code within a maximum of 8000 tokens. You should present the SVG code that best reconstructs the input image enclosed in triple quotes.

[16] https://app.endpoints.anyscale.com/
[17] https://platform.openai.com/docs/guides/gpt

Prompt 3. Used on GPT4 and CodeLlama for Text-to-SVG Generation: You are a helpful assistant assisting researchers in generating SVG code from textual descriptions. You will be provided with details to guide your SVG creation. Your task is to write SVG code that accurately represents the given textual information to the fullest extent possible. You are committed to solving the task of SVG generation for a robust system, so always strive to produce the best SVG code you can. Feel free to use multiple paths and any necessary shapes, colors, or lines to generate compilable SVG code within a maximum of 9000 tokens. The goal is to ensure the resulting SVG, when rasterized, best represents the described content. Respond only with the SVG code, enclosed in triple quotes, that directly corresponds to the provided textual description. Avoid adding any explanation or commentary.

IconShop IconShop [94] uses a transformer-based architecture to encode path commands and learn to model SVG path sequences autorregressively. It has shown excellent results in simplified icon scenarios and provides a good solution to Text-to-SVG generation by extending the FIGR-SVG dataset with captions. We have access to their dataset and original splits and have trained our model on that data using a pre-trained checkpoint (trained on SVG-Stack). We have extracted the results from IconShop and included them here to compare our method.

10. Additional Experiments and Results

10.0.1. Image-to-SVG Results

We show additional Image-to-SVG results from StarVector. Figures [13 - 18] show substantial qualitative samples generated by StarVector on all the proposed datasets. All results are computed in the test sets. We can observe the weaknesses and strengths of our model. Simplified datasets (Figures 16) are near-perfectly converted to SVG. In the case of icons, in Figure 18, sometimes the model runs out of SVG code tokens, and the image is incomplete. Results on SVG-Emoji 17 show impressive performance in estimating the shape's color and semantics. However, it lacks fine-grained and accurate positioning of objects, i.e., in some examples, the model loses track of the coherent position and form of shapes. These problems result from insufficient emoji samples, i.e., less than 10,000 training examples. This problem can be alleviated by scaling up the current model in the number of parameters (currently 1.4 billion), training data for pre-training, and computing resources.

Figure 13. **Image-to-SVG Results on SVG-Stack**. We present vectorizations of StarVector-1B on the test set of SVG-Stack. Left is input raster image, right is the SVG image (in SVG format).

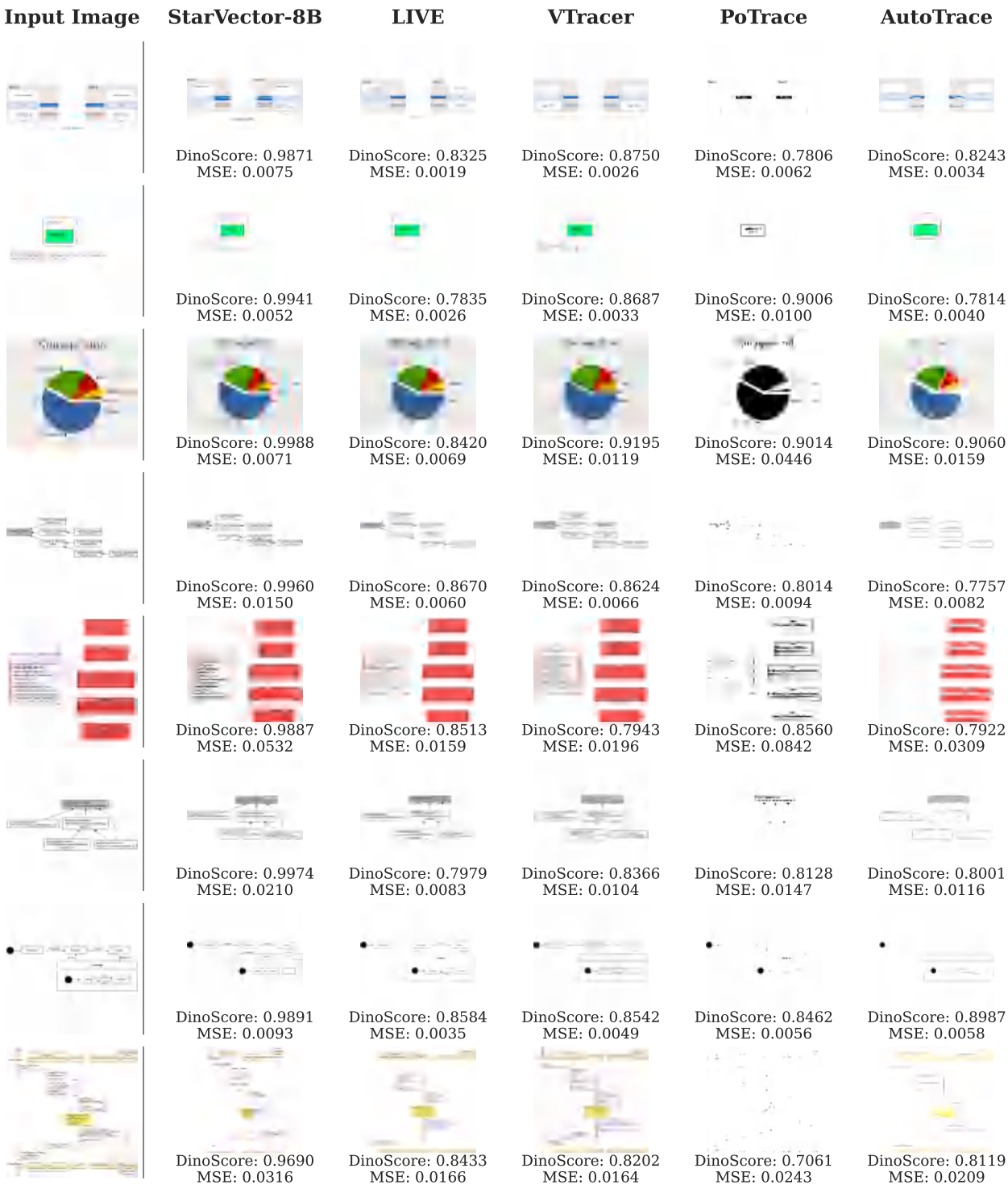


Figure 14. We compare the results from StarVector-8B with those from the most powerful baselines. Notably, StarVector is *the only* method capable of producing acceptable results that preserve both structural integrity and textual content by utilizing a variety of SVG primitives. In contrast, other methods tend to generate blobs and curves that merely attempt to fit the structure and color of the original image. We present two metric scores for each sample: DinoScore and MSE. MSE consistently yields higher scores for other methods, as they focus on fitting vectors to the image as accurately as possible. While StarVector may not achieve perfect reconstruction, its results are preferred for their semantic fidelity. This highlights the limitations of MSE and the importance of DinoScore in capturing these aspects.

Figure 15. **Image-to-SVG results on SVG-Fonts** test set. Results are remarkably good, obtaining perfect font reconstructions. Intricate details are preserved. This is because the dataset is very large, above 1M samples. This shows that if having access to a large dataset, StarVector can learn high-quality SVG generation.

Figure 16. **Image-to-SVG results on SVG-Fonts simplified** test set.

Figure 17. **Image-to-SVG results on SVG-Emoji** test set. Results are mostly wrong in this benchmark, due to the small training dataset of approximately 8k examples.

Figure 18. **Image-to-SVG results on SVG-Icons** test set.

Comparing parameter count of models . The number of learnable parameters in deep learning based models often correlates with performance. Pre-LLM models like Im2Vec and DeepSVG use significantly fewer parameters (up to 5M) compared to StarVector and GPT-based models, which operate in the billions. While pre-LLM models can produce accurate results, they lack the generalization ability of LLM-based approaches. Comparing StarVector with GPT models reveals that high-fidelity SVG generation is achievable with just 1B parameters, whereas GPT-4V lacks specific training for this task. Future models will likely incorporate SVG data in training, but current results already demonstrate that LLM-based approaches offer superior generalization and scalability for SVG generation, at the cost of utilizing more parameters.

Context Length Limitation. The model's architecture imposes a clear limitation on context length, which significantly impacts training and testing data pipelines, as well as the skills the model can learn. Our experiments show that the model scales effectively with increasing context lengths, from 8k to 16k, indicating that this limitation must be addressed with techniques for handling longer contexts—an area LLMs are expected to improve. For fair comparisons, we restricted our benchmark tests to a context length of 8k and evaluated all baselines within this setting. However, the benchmarks also provide versions with longer context lengths to assess future models, as increased length generally correlates with more complex SVGs. We did not observe substantial differences in scores between the 4k and 10k token settings, primarily because the data in our benchmark can typically be represented using an average of 3k tokens.

Limitations on Complex SVG Structures. StarVector encounters challenges with complex SVG structures, intricate shapes, and detailed illustrations primarily due to limitations in its architecture. Currently, the model's image encoder handles images by simply padding and resizing them to fixed dimensions of 224 or 384 pixels. This approach may not adequately capture the nuances of complex diagrams. A potential improvement would be to implement a dynamic image processing system akin to those found in newer Vision-Language Models (VLMs), which could enhance the model's ability to interpret and generate intricate SVGs more effectively. Additionally, improving data cleaning processes is crucial, as the model sometimes produces hallucinated information due to noise in the input data, such as URLs or base64-encoded images. Addressing these issues through architectural enhancements and more robust data preprocessing could significantly improve StarVector's performance on complex SVG tasks.

Generalization to Non-Standard SVGs. StarVector's ability to generalize to non-standard SVGs—those not represented in its main training distribution—poses a significant challenge. While the model performs well on common styles and primitives encountered during training, it struggles with more unique or unconventional SVGs. This is primarily due to the model's training data, which tends to focus on widely used shapes and designs. As a result, StarVector may exhibit a bias towards these common styles, leading to suboptimal performance when faced with SVGs that feature unusual structures or less frequent elements.

To assess StarVector's generalization capabilities, we evaluated its performance on various datasets that include non-standard SVGs. The results indicate that while the model can produce reasonable outputs for some non-standard examples, it often falls short in accurately capturing the intricacies of less familiar styles. This limitation suggests that the model's training set lacks sufficient diversity to encompass the full range of potential SVG designs.

To address these concerns, future work should focus on expanding the training dataset to include a wider variety of SVG styles and structures. Incorporating data from niche applications and artistic domains could enhance the model's ability to generate SVGs across a broader spectrum of design elements. Additionally, techniques such as domain adaptation and transfer learning could be explored to improve generalization to non-standard SVGs, allowing StarVector to adapt more effectively to unfamiliar inputs.

10.1. Ablation Studies

We performed ablations on the image encoder type, the data augmentation pipeline, inference techniques, and generation parameters. Most of our ablations were performed on the StarVector-1B model for faster iteration, and we empirically find they work well on the larger StarVector-8B.

Image Encoder Ablation. The choice of image encoder for the problem of Image-to-SVG is highly impactful, as it determines how well visual information from raster images can be preserved in a representation suitable for precise reconstruction in the SVG space. We ablated the visual encoders by replacing them with VQGAN [23], ConvNext [46], and CLIP ViT-B/32 [62], in our StarVector-1B proposed architecture. This setup evaluates three commonly used approaches in visual representation learning [23, 62, 73]. In our experiments, CLIP consistently outperformed across all metrics for various datasets (see Table 9). Figures [19–22 further illustrate how VQGAN and ConvNext tend to lose local details during generation, even while maintaining semantic relevance.

Sampling technique	SVG-Fonts			SVG-Emojis			SVG-Icons			SVG-Stack		
	LPIPS ↓	SSIM ↑	MSE ↓	LPIPS	SSIM	MSE	LPIPS	SSIM	MSE	LPIPS	SSIM	MSE
Greedy	0.019	0.969	0.013	0.251	0.731	0.071	0.059	0.912	0.028	0.157	0.797	0.067
+ Beam Search (B=5)	0.018	0.970	0.012	0.250	0.732	0.070	0.058	0.913	0.027	0.156	**0.798**	**0.066**
Nucleus Sampling (T=0.5)	**0.013**	**0.976**	**0.008**	**0.202**	**0.778**	**0.051**	**0.043**	**0.923**	**0.022**	**0.153**	0.785	0.072
Nucleus Sampling (T=1.0)	0.015	0.975	0.009	0.244	0.742	0.067	0.053	0.917	0.025	0.161	0.786	0.069
+ Beam-Search (B=5)	0.034	0.948	0.027	0.244	0.742	0.068	0.065	0.913	0.027	0.195	0.766	0.089
+ Beam-Search (B=10)	0.040	0.943	0.031	0.251	0.742	0.072	0.071	0.910	0.028	0.175	0.762	0.079

Table 7. **Ablation study on sampling strategies**. We experimented using greedy decoding and added a beam search with B=5. We test nucleus sampling [31] using top p=0.9, with temperatures T=0.5 and T=1.0. The two final rows describe beam search with nucleus sampling at T=1.0. See `huggingface.com/blog/how-to-generate` for reference on these sampling techniques.

Pre-training on SVG-Stack. Pre-training on the SVG-Stack is highly beneficial for the downstream datasets with small data. Table 8 shows the uplift on all the metrics for different datasets. Qualitatively, we can also see that pre-training helps the model to identify the nuanced details from the images. For the case of SVG-Emoji, pre-training is a vital requirement, as it overfits without it due to limited data. Figure 17 shows that the model relies on colors and shapes to generate the SVG.

Ablation on Generation Hyperparameters. We explore the impact of different generation hyperparameters on the StarVector-1B model. After an initial exploration to empirically determine the most relevant hyperparameters, we focus our ablation on these. We find that temperature and the number of beams in beam search significantly affect performance. The model is evaluated across various configurations (see Table 7 and Figure 23). Our results show that a beam search size of 5 achieves the best outcomes, albeit with increased memory usage and runtime. Similarly, nucleus sampling with a top-p of 0.9 and a temperature of 0.5 delivers the best overall performance.

10.2. Text-to-SVG Results

Figures [25 - 28] show additional qualitative results of StarVector when performing the task of text-conditioned SVG generation, performed on SVG-Stack and FIGR-SVG test sets. Our samples show reasonable effectiveness at this task, consistently grasping features like colors, shapes, and semantic concepts. However, sometimes some details required in the prompt are lost, e.g., an exact number of circles, shapes inside other shapes, or the direction of arrows. In some cases, some vector graphics shapes lose coherence, which we attribute to our model's current scale in terms of model parameters and context length. We suspect that these mistakes are due to the limited quality of the textual descriptions, sometimes lacking precision and grounding on the

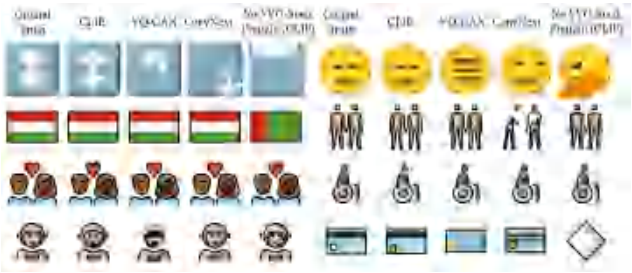


Figure 19. **Ablation of Image Encoders** Image vectorization results using different visual encoders on SVG-emoji test set. CLIP is the image encoder that delivers the best results, whereas VQ-GAN and ConvNet often miss relevant semantics of the image. No SVG-Stack Pretrain (CLIP) refers to an ablation where we use CLIP out of the box, without unfreezing its weights.

Figure 20. **Ablation of Image Encoders** Image vectorization results using different visual encoders on SVG-Stack test set. CLIP offers the best results. VQ-GAN and ConvNet often miss relevant semantics of the image.

SVG images. See Figure 24 for successful cases of Text-to-SVG generation on SVG-Stack. Figure 29 highlights some failure modes of StarVector-8B. These figures illustrate the impact of different generation temperatures. We rank the outputs generated at different temperatures based on their CLIP Score in relation to the text instruction.

Nevertheless, the StarVector approach of using LLMs for SVG code generation is the only method among baselines that allows us to create diverse vector graphics unre-

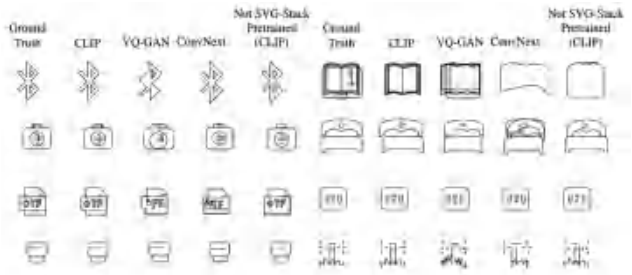


Figure 21. **Ablation of Image Encoders** Image vectorization results using different visual encoders on SVG-Icons test set. CLIP brings the best visual results. VQ-GAN and ConvNext are not able to capture correctly the details for correct vectorization. No SVG-Stack Pretrain (CLIP) refers to an ablation where we use CLIP out of the box, without unfreezing its weights. Notably, better results are obtained when training the CLIP image encoder on SVG-Stack.

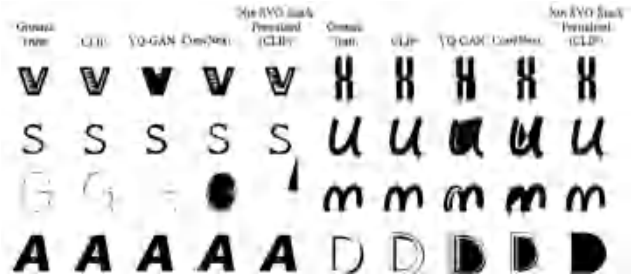


Figure 22. **Ablation of Image Encoders** Image vectorization results using different visual encoders on SVG-Fonts test set. As in the other datasets tested, CLIP brings the best visual results, as others are not able to provide perfect vector reconstruction when intricate details are present. No SVG-Stack Pretrain (CLIP) refers to an ablation where we use CLIP out of the box, without unfreezing its weights. Notably, better results are obtained when training the CLIP image encoder on SVG-Stack.

strainedly, paving the way for more challenging and intricate designs.

10.3. Results on SVG-Diagrams

Figure 2 presents the results of StarVector-8B, along with comparisons to LIVE, VTracer, Potrace, and AutoTrace. StarVector-8B is the only approach that produces plausible results, as it effectively leverages appropriate SVG primitives. DinoScore aligns well with this visual assessment, accurately reflecting the quality of StarVector-8B's outputs.

In contrast, MSE consistently favors other baselines despite their limitations. This is because MSE prioritizes exact pixel matching, favoring models designed to fit curves and colors to the input image. However, these baselines fail to preserve the semantics of the original diagrams, resulting in outputs where the meaning and structure are completely lost.

Table 8. **Results of SVG Data Augmentation.** We ablate both our data augmentation pipeline and the use of a pretraining stage with SVG-Stack. These experiments are conducted on smaller datasets that are more susceptible to overfitting, using the StarVector-1B model. Vanilla refers to the StarVector model trained directly on the given dataset without SVG-Stack pretraining. Next, we introduce our data augmentation pipeline. Finally, we initialize training from an SVG-Stack pretrained checkpoint and fine-tune on the given dataset. The "+" symbol indicates that the methods from the previous rows are also included.

	SVG-Emojis			SVG-Icons		
Method	**LPIPS** ↓	**SSIM** ↑	**MSE** ↓	**LPIPS**↓	**SSIM**↑	**MSE**↓
StarVector (vanilla)	0.355	0.683	0.108	0.104	0.845	0.047
+ Data Augmentation	0.329	0.706	0.097	0.057	0.905	0.029
+ SVG-Stack Pretrain	**0.225**	**0.748**	**0.061**	**0.057**	**0.894**	**0.031**

Table 9. **Ablation of Image Encoders.** We ablate different image encoders with StarVector-1B, namely CLIP ViT-B/32 [62], VQ-GAN [23], and ConvNext [46]. We experiment with training experiments on SVG-Fonts and SVG-Emojis datasets. CLIP gives the best results on all reconstruction metrics.

	SVG-Fonts			SVG-Emojis		
Encoder	**LPIPS** ↓	**SSIM** ↑	**MSE** ↓	**LPIPS** ↓	**SSIM** ↑	**MSE** ↓
CLIP	**0.026**	**0.955**	**0.021**	**0.202**	**0.778**	**0.051**
VQGAN	0.092	0.854	0.072	0.345	0.688	0.099
ConvNext	0.085	0.854	0.073	0.311	0.708	0.088

10.4. Analysis of SVG Primitives

This section examines how StarVector leverages SVG primitives to produce more compact and semantically accurate SVGs. In contrast to prior models constrained to using only *path* primitives, StarVector effectively utilizes the entire range of SVG primitives, including parametrically defined shapes, gradients, and text elements.

This enhanced capability stems from its ability to operate directly within the SVG code space, facilitated by its multimodal, transformer-based architecture [2, 45], which integrates visual and textual inputs. StarVector generates SVG code that closely resembles the input raster image while maintaining semantic awareness, enabling the use of symmetry, parametric shapes, and text. Prior methods, limited to first-order *path* primitives, lack this semantic understanding, resulting in less compact and less expressive SVG representations.

Qualitative Analysis of SVG Primitives Table 10 presents tests conducted on StarVector-8B and VTracer using simple designs composed of basic shapes such as circles, rectangles, and triangles, with variations in color, transparency, and levels of overlap. StarVector-8B demonstrates the ability to precisely identify the primitives that

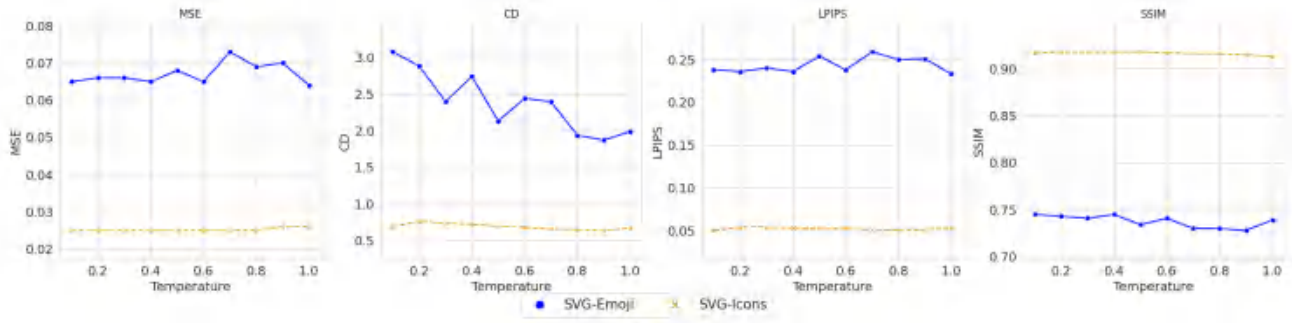


Figure 23. **Ablation study on sampling temperature**. We tested the performance impact of StarVector-1B when changing the sampling temperature. Results are computed for SVG-Emoji and SVG-Icons validation sets.

make up each design, producing visually accurate results while maintaining compact, concise, and interpretable SVG representations.

SVG Tag Distribution Here, we show how StarVector can generate complex SVGs using the full syntax of the SVG language, in contrast with most of the literature methods, which are restricted to using only the *path* command. Figure 30 displays the distribution of SVG tags in the SVGs generated by StarVector along with the distribution in the SVG-Stack dataset, showcasing the strength of our method in using SVG tags and syntax in a similar way to the original in-the-wild dataset. We have computed the exact statistics on previous methods and found that they cannot come close to StarVector in this metric, as they are limited to using *paths* and basic primitives. The effective usage of the large array of SVG tags and syntax makes our method the first SVG model to support these complexities.

10.5. Human Evaluation

We conducted a human evaluation to compare the outputs of StarVector-8B, our best model, with those of the most powerful baselines. Participants were selected from diverse backgrounds and carefully screened for conflicts of interest, with none of the key authors involved. The evaluation was performed through a web interface (shown in Figure 31) that provided anonymized outputs and randomized sample presentations to prevent pattern recognition or bias.

The results, presented in Figure 5, demonstrate a strong preference for StarVector-8B across all settings, especially in SVG-Diagrams tasks. This highlights a disconnect between pixel-based metrics (e.g., MSE, SSIM) and human visual perception of SVGs. While baseline models often prioritize pixel-perfect reconstruction, human evaluators preferred StarVector's sharp, well-defined shapes and its effective use of primitives (Figure 30).

Spearman correlation analyses between model metrics and human evaluations further emphasize this gap. MSE shows weak correlations (0.0596 and -0.1002), indicating its inadequacy as a predictor of human preferences. In contrast, DinoScore exhibits significantly stronger correlations, with values of -0.6193 and 0.6214. Moreover, a robust correlation of 0.7577 between differences in DinoScore and human evaluation scores highlights DinoScore as a more reliable metric for assessing SVG quality in alignment with human judgment.

10.6. Comparing StarVector with Baselines

Here, we discuss the results of each baseline individually and compare them to our proposed approach.

1. **DeepSVG** [14] is an elegant approach to learning a latent variable model for SVG. It proves effective at learning the task for the *simplified* datasets (Figure 9). It can accurately represent corners and edges. However, it only works in simplified datasets. This limitation restricts it from being a suitable solution in real applications.

2. **Im2Vec** [67] proposes a training procedure that does not require having SVG ground truth. It uses only pixel-based reconstruction loss with the input image, finding the optimal SVG parameters using a differentiable rasterizer like DiffVG [43]. This framework is appealing, as it aims to be used in images without SVG supervision. However, it requires hundreds of epochs with a reduced dataset to overfit the model to those examples, only working on modeling *training* examples, as seen in [67]. Im2Vec results on the datasets presented SVG-Bench are quite poor as it has bad generalization. Therefore, qualitative samples are not presented.

3. **GPT-4 Vision** [56] excels at capturing the semantics of images and generating captions that accurately describe them. The SVG generated from this description

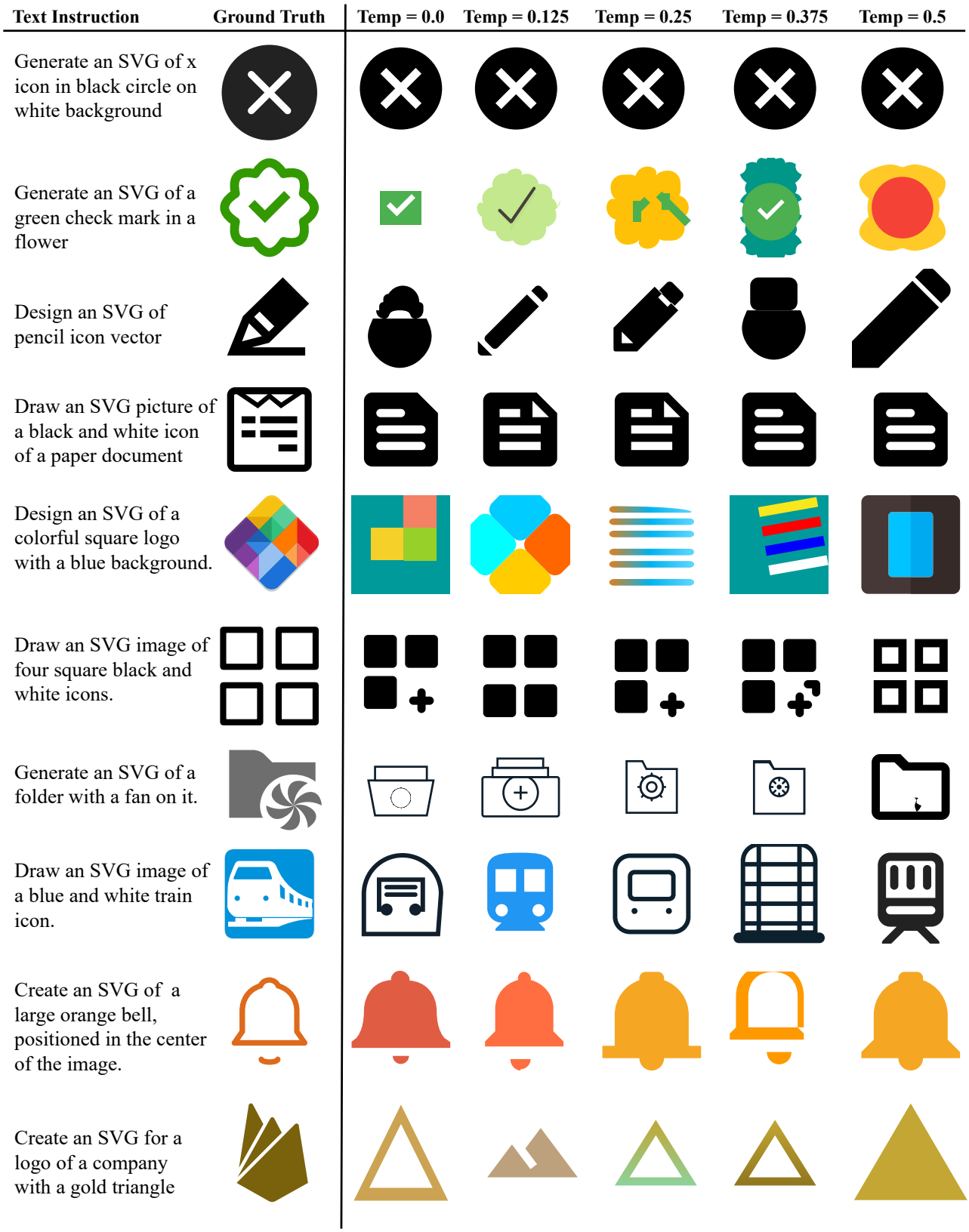


Figure 24. **Text-to-SVG Results**. We show successful Text-to-SVG results using StarVector-8B. We sample 5 different temperatures as an ablation, showing the sensitivity of this parameter during generation. Results are presented in SVG (not raster images)

Figure 25. **Text-to-SVG Generation** results using StarVector-1B on SVG-Stack test set (i).

Figure 26. **Text-to-SVG Generation** results using StarVector-1B on SVG-Stack test set(ii).

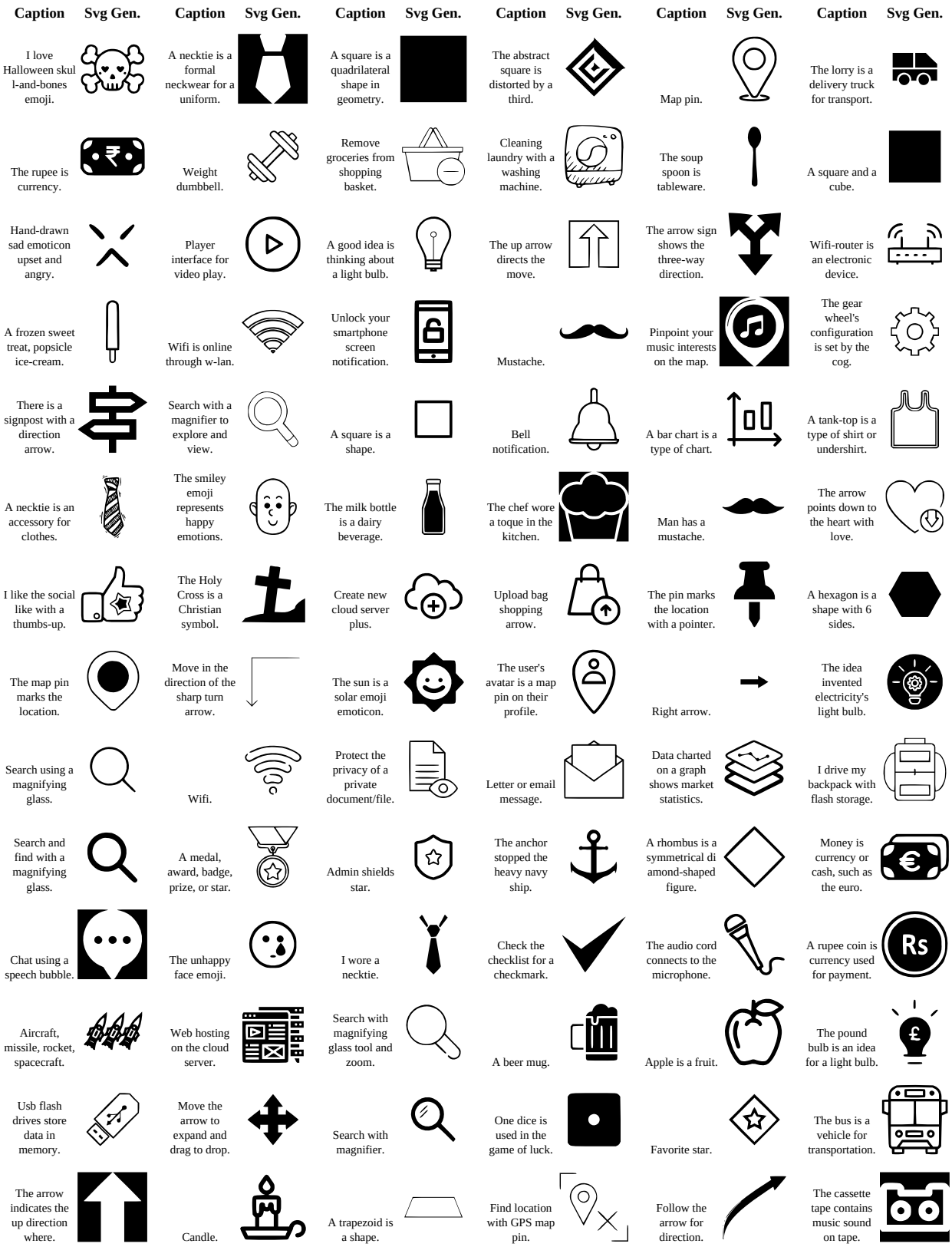


Figure 27. **Text-to-SVG generation** on FIGR-SVG test set (i).

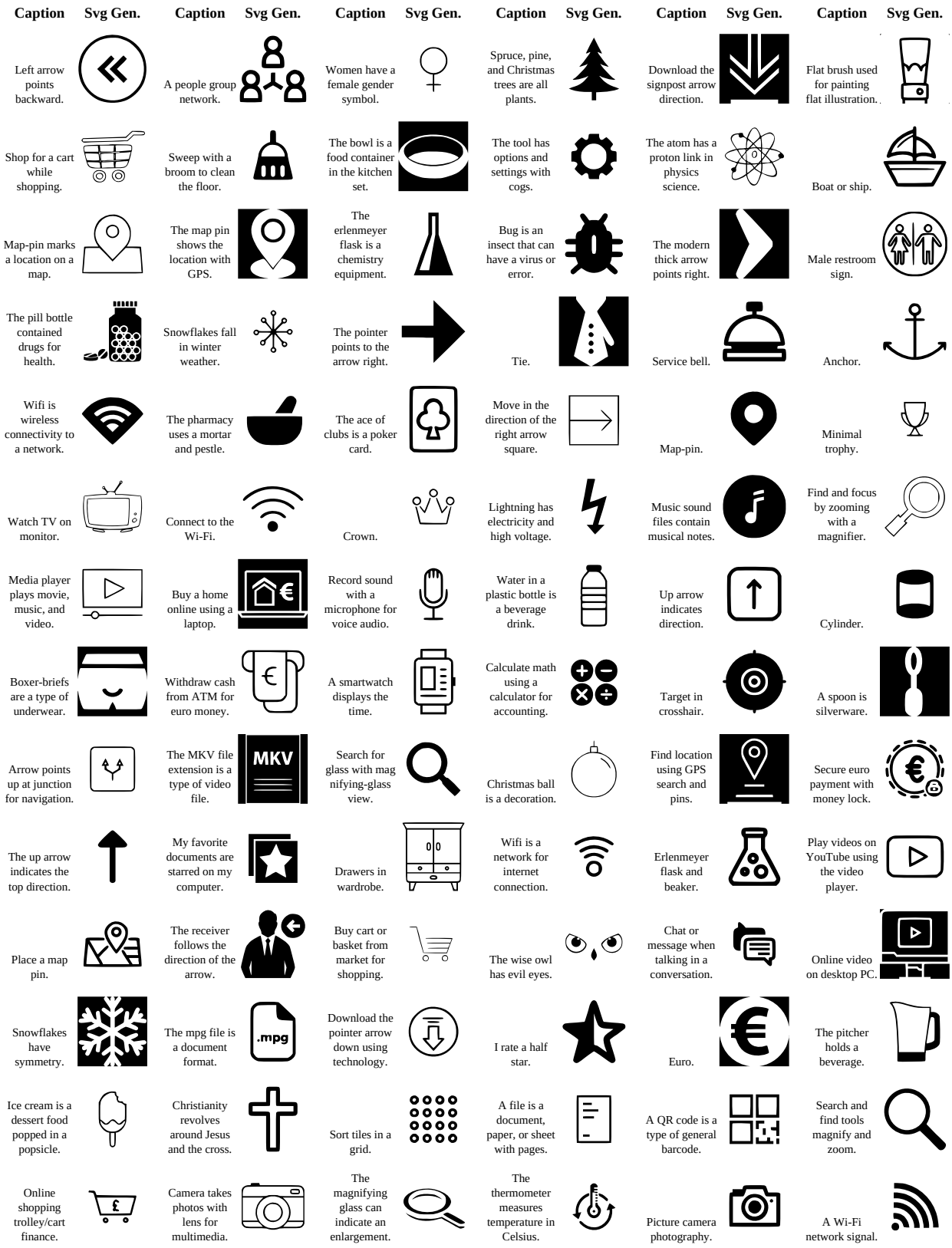


Figure 28. **Text-to-SVG generation** on FIGR-SVG test set (ii).

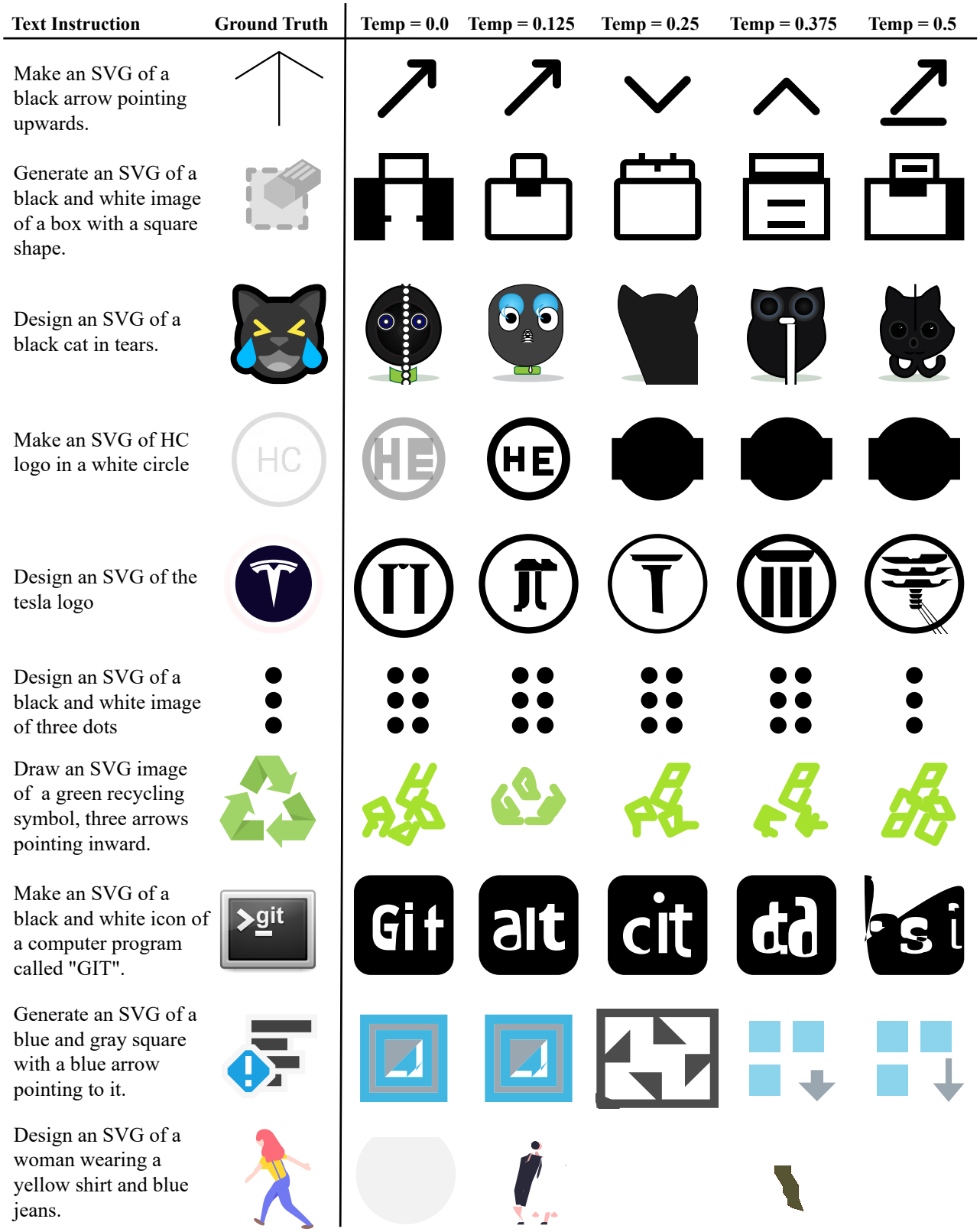
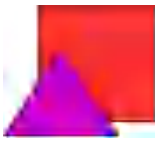


Figure 29. **Text-to-SVG Results**. We show failure Text-to-SVG results using StarVector-8B (cherry picked examples that show limitations). We sample 5 different temperatures as an ablation, showing the sensitivity of this parameter during generation. Results are presented in SVG (not raster images)

Table 10. **Usage of SVG Primitives.** Image vectorization results of StarVector and VTracer applied to images containing basic shapes, such as circles, rectangles, and polygons, with varied colors and transparencies. The leftmost column shows the input images prompted for vectorization, and other columns show the output SVG code, with the SVG primitives in red color. StarVector accurately identifies and generates SVG code for each primitive, preserving their distinct characteristics. In contrast, VTracer relies on the `path` primitive, resulting in SVG code that captures the input image in terms of pixels, with less fidelity to individual shapes. Due to the length of VTracer's SVG output, only the initial lines are shown. VTracer serves as a baseline model, representative of other baselines, which are omitted for space but exhibit similar behavior, primarily using `path` without shape recognition.

Test Example	StarVector	VTracer

	`<svg width="150" height="150" xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink">`
	`<rect x="35" y="0" width="114" height="132" style="fill:rgb(255,50,50);stroke-width:1;stroke:rgb(0,0,0)"/>`
	`<polygon points="56.25,49.5 112.5,147.75 0,147.75" style="fill:rgb(200,0,200);stroke-width:1;stroke:rgb(0,0,0)"/>`
	`</svg>`
		`<svg version="1.1" xmlns="http://www.w3.org/2000/svg" width="530" height="460">`
		`<path d="M0 0 C174.9 0 349.8 0 530 0 C530 151.8 530 303.6 530 460 C355.1 460 180.2 460 0 460 C0 308.2 0 156.39999999999998 0 0 Z " fill="#FE3232" transform="translate(0,0)"/>`
		`<path d="M0 0 C4.06429570258797 3.6330418563035494 6.775718865918918 7.966461775462079 9.71484375 12.50390625 C10.273576812744153 13.357300872802739 10.832309875488278 14.210695495605478..." fill="#A95869" transform="translate(343,60)"/>`
		`<path d="M0 0 C40.26000000000005 0 80.52000000000001 0 122 0 C123.16960912291928 66.0224184213402 123.16960912291928 66.0224184213402 123.416015625 93.876953125 C123.47276957931317..." fill="#A95869" transform="translate(343,60)"/>`
		...
		`</svg>`

	`<svg version="1.1" id="Layer_1" xmlns="http://www.w3.org/2000/svg" xmlns:xlink="http://www.w3.org/1999/xlink" x="0px" y="0px" width="64px" height="64px" viewBox="0 0 64 64" enable-background="new 0 0 64 64" xml:space="preserve">` `<ellipse fill="none" stroke="#CD2A2A" stroke-width="2.5" stroke-miterlimit="10" cx="32.333" cy="32.083" rx="29.333" ry="31.25"/>` `<polygon fill="#CD2A2A" stroke="#CD2A2A" stroke-width="0.25" stroke-miterlimit="10" points="32.333,3.833 55.167,46.333 9.5,46.333 "/>` `<line fill="none" stroke="#000000" stroke-width="0.25" stroke-miterlimit="10" x1="0" y1="0.167"x2="64"y2="0.167"/>` `</svg>`	`<svg version="1.1" xmlns="http://www.w3.org/2000/svg" width="1196" height="1140">` `<path d="M0 0 C394.68 0 789.36 0 1196 0 C1196 376.20000000000005 1196 752.4000000000001 1196 1140 C801.3199999999999 1140 406.64 1140 0 1140 C0 763.8 0 387.5999999999999 0 0 Z " fill="#FEFEFE" transform="translate(0,0)"/>` `<path d="M0 0 C1.8671875 0.24609375 1.8671875 0.24609375 4 1 C5.1328125 2.722656250000071 5.1328125 2.722656250000071 6.125 5.0625 C8.342476699064719 9.859072675207557 10.955923466443437 14.196311705833779 13.8125 18.625 C17.369173047863796..." fill="#A95869" transform="translate(343,60)"/>` `<path d="M0 0 C1.3488281421188617 2.6976562842377234 0.8033826481469646 4.151643338387942 0 7 C-0.6599999999999682 7.329999999999984 -1.3199999999999932 7.660000000000025 -2 8 C-2.6555511917654258 10.527332350510733 -2.6555511917654258 10.527332350510733 ..." fill="#A95869" transform="translate(343,60)"/>` ... `</svg>`

	`<svg width="300" height="300" xmlns="http://www.w3.org/2000/svg" version="1.1">`
	`<rect x="110" y="10" width="160" height="230" fill="pink" stroke="red"/>`
	`<circle cx="160" cy="120" r="120" fill="tan" stroke="green"/>`
	`<polygon points="110,20 280,125 175,280 5,180" fill="blue" fill-opacity="0.25" stroke="blue" stroke-width="2" />`
	`</svg>`
		`<svg version="1.1" xmlns="http://www.w3.org/2000/svg" width="910" height="934">`
		`<path d="M0 0 C300.3 0 600.6 0 910 0 C910 308.22 910 616.44 910 934 C609.7 934 309.4 934 0 934 C0 625.78 0 317.55999999999995 0 0 Z " fill="#928299" transform="translate(0,0)"/>`
		`<path d="M0 0 C1.4849999999999994 0.9900000000000091 1.4849999999999994 0.9900000000000091 3 2 C3 2.659999999999968 3 3.32000000000005 3 4 C3.7296093749999955 4.103124999999977 ..." fill="#A95869" transform="translate(343,60)"/>`
		`<path d="M0 0 C1.32000000000005 0.6599999999999966 2.6399999999999864 1.3200000000000003 4 2 C3.009999999999991 2 2.019999999999982 2 1 2 C1 2.6599999999999966 1 3.3200000000000003" fill="#A95869" transform="translate(343,60)"/>`
		...
		`</svg>`

	`<svg viewBox="0 0 100 100" xmlns="http://www.w3.org/2000/svg">`
	`<circle cx="50.5" cy="39.5" r="40" fill="#60f0ff" />`
	`<ellipse cx="50" cy="39" rx="50" ry="20" fill="#2810ff" />`
	`<polygon points="10,100 50,42 90,100" fill="green" />`
	`</svg>`
		`<svg version="1.1" xmlns="http://www.w3.org/2000/svg" width="510" height="520">`
		`<path d="M0 0 C168.3 0 336.6 0 510 0 C510 171.6 510 343.2 510 520 C341.7 520 173.39999999999998 520 0 520 C0 348.4 0 176.8 0 0 Z ..." fill="#2E17FE" transform="translate(0,0)"/>`
		`<path d="M0 0 C0.6599999999999682 0 1.3199999999999932 0 2 0 C3.1463165283203125 1.3810577392578693 3.1463165283203125 1.3810577392578693 4 4.428955078125 3.356689453125 C4.916970977783194 " fill="#2E17FE" transform="translate(0,0)"/>`
		`<path d="M0 0 C1.5393356364577242 2.844848138263643 2.426629556693727 5.489600992013067 3.1875 8.625 C8.714349606195654 27.37779319117132 28.666518702768336 42.668449172855844 45 52 C77.0038243523029 ..." fill="#2E17FE" transform="translate(0,0)"/>`
		`<path d="M0 0 C0.33000000000409 0 0.6599999999999682 0 1 0 C1 101.31 1 202.62 1 307 C-16.159999999999968 307 -33.31999999999999 307 -51 307 C-69.10137393182612 285.06539394143425 -69.10137393182612 285.06539394143425 -73.55859375 275.48046875 ..." fill="#2E17FE" transform="translate(0,0)"/>`
		...
		`</svg>`

is valid and effectively incorporates semantic concepts along with the accurate colors of the input image into the SVG code (see Figures [9 - 12]). However, it falls short in terms of reconstruction fidelity, as GPT-4V was not specifically trained for reconstruction tasks, making these results predictable.

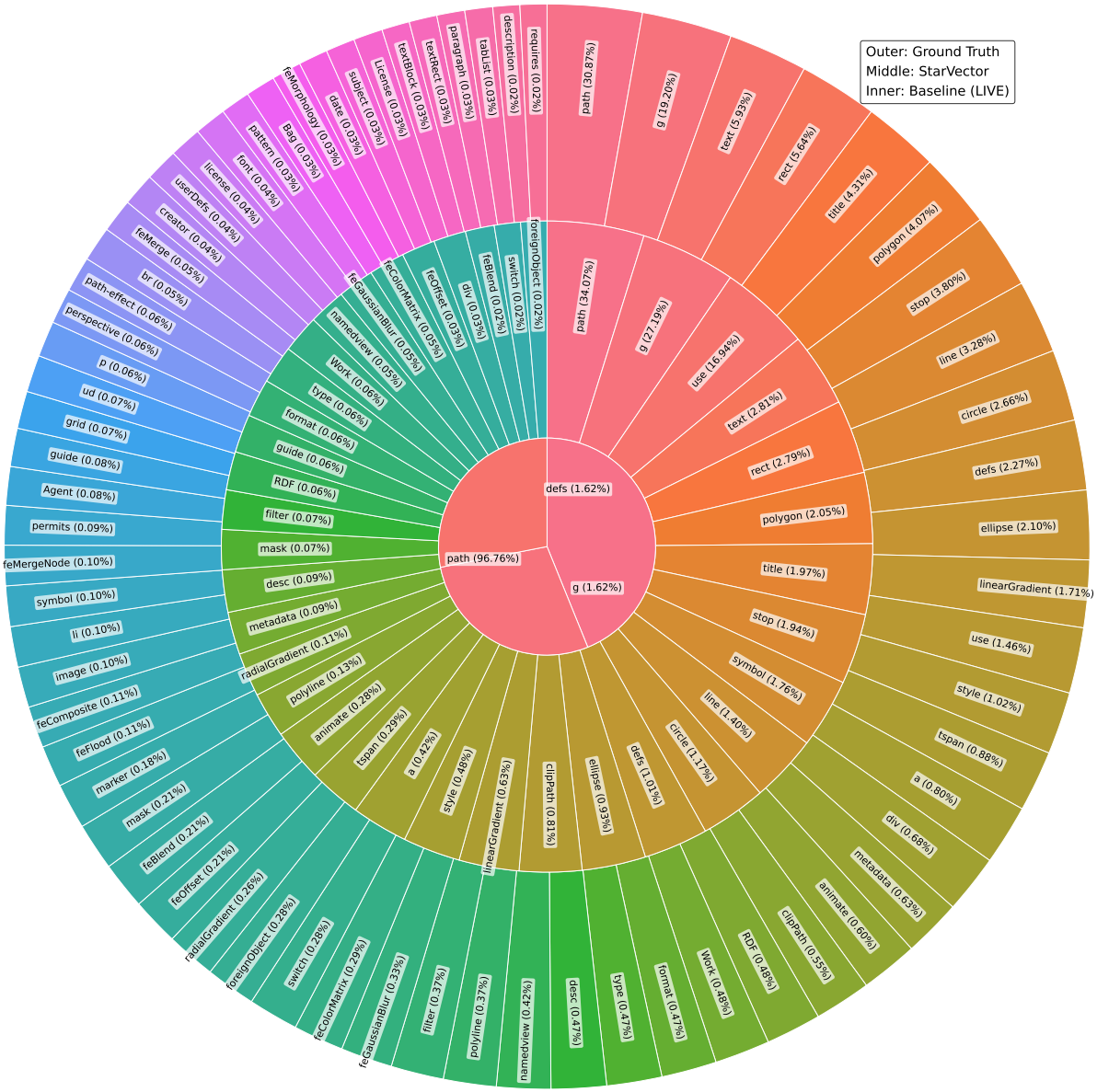


Figure 30. **Distribution of SVG Primitives and SVG Tags.** We show the frequency of SVG tags that appear in the ground truth of SVG-Stack dataset (outer ring), compared to the frequency of tags generated by StarVector (middle ring), and compared to our most performant baseline (LIVE). For visualization purposes, we apply a logarithmic scaling over the counts and show the base percentage in parentheses. StarVector generates SVGs that contain tags with a similar distribution to the ground truth. The baseline is limited to `paths` or basic primitives.

4. **LIVE** achieves the best results in terms of pixel-based metrics like MSE, LPIPS, and SSIM (see Tables [1, 2, 3]). Using 32 paths, it effectively represents a wide range of images, making it highly versatile for vectorization tasks, including natural images, which StarVector cannot handle due to its specialized training. However, LIVE has notable limitations. It relies on a slow test-time optimization process (approximately 10 minutes per sample, using 32 paths, Table 3) to refine SVG outputs for good MSE scores, often introducing unwanted visual artifacts.

Additionally, its exclusive use of *path* primitives results in significantly larger SVG files (19k tokens), as shown in Table1. In contrast, StarVector produces compact (3k tokens) and professional-grade SVGs by leveraging a variety of SVG primitives beyond paths, achieving higher precision and efficiency.
Notably, we find that this method, and the broader family of differentiable techniques it belongs to, is unsuitable for generating images that require specific primitives, such as those in diagrams. Its performance on the SVG-Diagrams benchmark is poor, as illustrated in Figure 14.

5. **DiffVG** offers comparable results as the other baselines in terms of reconstruction metrics, and it can produce suitable SVG image vectorization with 32 paths (same as LIVE), as seen in Tables [1, 2, 3]. This tables also show that DiffVG produces large SVGs as seen in the number of tokens (Tokens column), approaximately 19k tokens, similar to LIVE. This means that files are extremely large compared to the ones of StarVector. As mentioned before, this is due to StarVector leveraging understanding and SVG primitieves. Nevertheless, this method is substantially faster than LIVE, requiring 30 seconds per sample.

6. **Image Processing Methods: VTracer, Potrace, Autotrace** *Previous image processing methods for Image-to-SVG are powerful*, excelling at fitting vector images to raster inputs with near-zero MSE while reliably capturing the shapes and colors of the original image. However, **we identify several shared limitations across these methods:** (1) lack of SVG file compression, as they often generate excessively long paths (see the Tokens column in Tables 1 and 2); (2) susceptibility to visual artifacts, especially with challenging patterns; and (3) poor performance in vectorizing diagrams, as illustrated in Figure 14. On the SVG-Diagrams benchmark, **VTracer**, **Potrace**, and **AutoTrace** struggle with producing high-quality results.
These methods perform best on images that can be segmented into distinct regions by color or texture but fail with complex patterns, such as small, closely spaced shapes or fine details. For example, in Figures 9 and 4, small polygons and intricate text are inadequately vectorized, with details often lost. All three methods are restricted to *path* primitives, limiting their ability to support features like optical character recognition or rendering text with the `<text>` tag. Although **Potrace** sometimes better preserves text compared to others, it still cannot recognize or encode it semantically.
Despite these limitations, these methods excel in generation speed, making them highly efficient for many tasks. As shown in Table 3, **VTracer** and **AutoTrace** can generate SVGs in under a second, while **Potrace** typically takes around 10 seconds. In comparison, StarVector requires over a minute, and other baselines can take anywhere from 10 to 20 minutes. While StarVector's semantic richness and compact outputs make it better suited for certain applications, these image processing methods demonstrate a clear advantage in scenarios where speed is critical.

7. **IconShop** achieves remarkable results on the SVG-FIGR dataset, as demonstrated in Table 4 and Figures of their original work [94]. However, it is not designed to handle SVG-Stack due to its restriction to modeling only the *path* primitive. StarVector outperforms IconShop, as shown in Tables 4, across metrics such as FID, FID CLIP, and CLIP Score. Qualitative examples further highlight StarVector's superior performance in Text-to-SVG generation within the FIGR-SVG dataset.

8. **LLMs for Code Generation.** *Methods utilizing LLMs to directly generate SVG code present appealing advantages.* In our evaluation, we assessed GPT-4, GPT-4V, CodeLlama, and our proposed StarVector approach. By leveraging the code space, these models can utilize various SVG primitives based on their understanding of raster images, including path shapes and higher-order primitives like circles and text. This capability enables applications in new domains, such as diagram generation, as demonstrated by the results in Figure 14. However, most LLMs have not been specifically trained for the Image-to-SVG task, which limits their performance. In contrast, StarVector outperforms other LLMs due to its dedicated architecture and tailored training methodology, excelling in both image understanding and SVG generation.
Upon reviewing the complete set of results, we conclude that the StarVector approach is the only deep learning-based Image-to-SVG model capable of achieving results comparable to those of VTracer, Potrace, and AutoTrace. Furthermore, StarVector paves the way for novel research avenues in vector graphics generation, enabling applications such as diagram generation, Text-to-SVG generation, and potentially enhanced editing and understanding of vector images.

10.7. Human Evaluation

We conducted a human evaluation to compare the outputs of StarVector-8B, our best model, with those of the most powerful baselines. Participants were selected from diverse backgrounds and carefully screened for conflicts of interest, with none of the key authors involved. The evaluation was performed through a web interface (shown in Figure 31) that provided anonymized outputs and randomized sample presentations to prevent pattern recognition or bias.

The results, presented in Figure 5, demonstrate a strong preference for StarVector-8B across all settings, especially in SVG-Diagrams tasks. This highlights a disconnect between pixel-based metrics (e.g., MSE, SSIM) and human visual perception of SVGs. While baseline models often prioritize pixel-perfect reconstruction, human evaluators preferred StarVector's sharp, well-defined shapes and its effective use of primitives (Figure 30).

Spearman correlation analyses between model metrics and human evaluations further emphasize this gap. MSE shows weak correlations (0.0596 and -0.1002), indicating its inadequacy as a predictor of human preferences. In contrast, DinoScore exhibits significantly stronger correlations, with values of -0.6193 and 0.6214. Moreover, a robust correlation of 0.7577 between differences in DinoScore and human evaluation scores highlights DinoScore as a more reliable metric for assessing SVG quality in alignment with human judgment.

11. StarVector Method

Here, we provide details on the StarVector architecture, training recipe, and generation process.

11.1. Architecture

11.1.1. Large Language Model

We consider several aspects when choosing the LLM to handle the SVG code generation. First, we require an LLM that can handle large token contexts during training, as SVG code samples are typically of long lengths (i.e., between 1,000-4,000 tokens for the most common SVG datasets but growing arbitrarily for much more complex vector graphics). Second, we need fast decoding during the generation of these large contexts. Finally, we would benefit from models that have been extensively pre-trained on general coding tasks to avoid early training costs.

Some prior works offer open-source models that fit these requirements. We explored the open-source families of models CodeGen [53], StarCoder [42] and StarCoder2 [49].

We empirically find the StarCoder family to be the most suitable choice for our requirements. StarCoder offers a pre-trained model with 1B parameters (`starcoderbase-1b`) and a context length of 8k tokens, making it ideal for smaller-scale experiments while maintaining efficient generation speeds. The model employs Multi-Query Attention and a Fill-in-the-Middle objective, trained on 1 trillion tokens, with a context window of 8192 tokens. Its compact size ensures compatibility with GPUs, facilitating data parallelism during training—a crucial benefit when fine-tuning all network parameters, including the memory-intensive image encoder.

The second generation, StarCoder2, extends the context length to 16,384 tokens, presenting an exciting avenue for exploring training on SVGs with longer context requirements. For this, we leverage the 7B parameter version (`starcoder2-7b`), which incorporates a sliding window attention mechanism of 4,096 tokens and was trained on over 3.5 trillion tokens of code using the same Fill-in-the-Middle objective [49]. This enhanced context capacity and training scale make it a promising candidate for scaling SVG-based experiments.

These two types of LLMs define the backbones of our two StarVector variants. StarVector-1B is based on the `starcoderbase-1b` architecture and weights, while StarVector-8B is based on the `starcoder2-7b` architecture and weights.

11.1.2. Image Encoder

Our image encoding pipeline computes the images' feature representations using a backbone image encoder and aligns them to the LLM via the Adapter module (see Figure 3). State-of-the-art image encoders are typically focused on natural images. However, our data contains images of logotypes, icons, fonts, or emojis, which usually contain no background (which we set to white) and mostly constant colors.

Note that the image encoder is used exclusively for the Image-to-SVG task and is not employed during the Text-to-SVG task. For Image-to-SVG, images must be projected into a representation with the same dimensionality as that of the LLM. We train the model to enable the LLM to ingest these representations and generate SVG code sequentially.

To choose the best encoder, we draw inspiration from the success of pre-trained encoder backbones in downstream computer vision tasks such as classification [62], retrieval, and generation [23], including both convolutional and transformer-based models. Specifically, we experiment with CLIP ViT-B/32 [62], ConvNext [46] (pre-trained on LAION-2B [75]), and VQGAN [23], which we pre-train on an image reconstruction task using raster images from SVG-Stack. For CLIP, we have $L_v = 257$ embeddings, including the CLS token. For VQGAN, we use the pre-quantization layers and flatten them to obtain $L_v = 196$ embeddings. For ConvNext, we flatten the last activation map to get $L_v = 49$ embeddings.

We explore several image encoders based on different

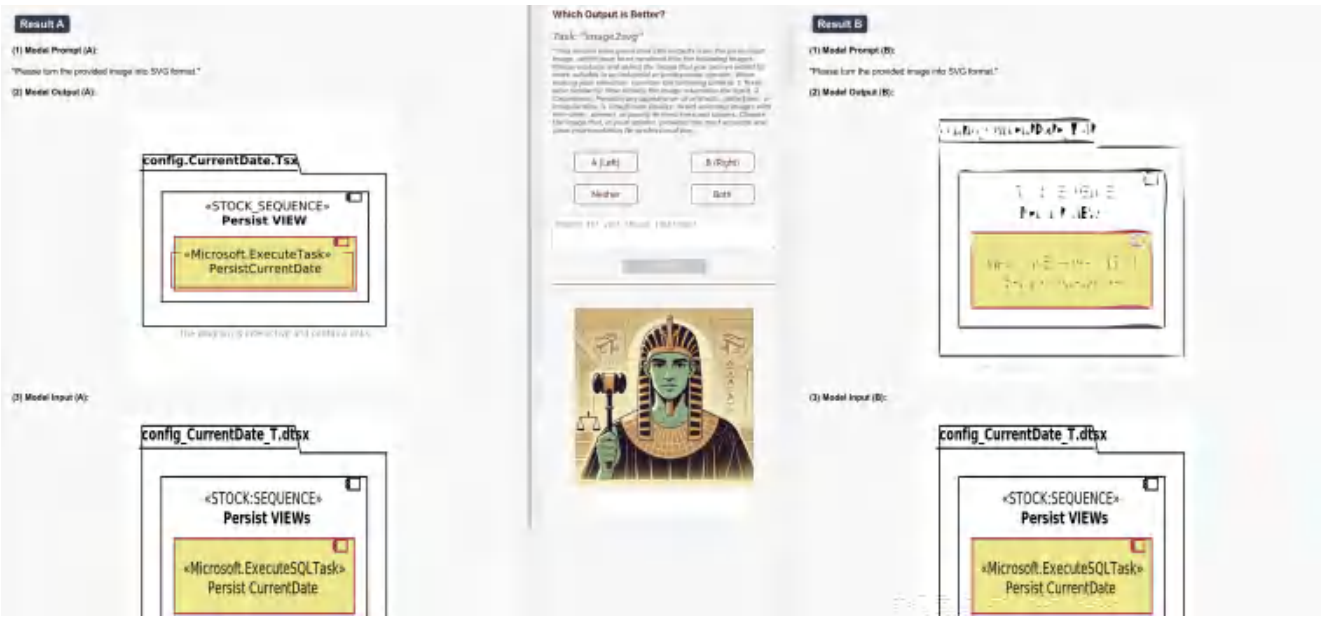


Figure 31. The web interface used during the human evaluation.

paradigms. VQGAN [23] is based on learning to project images to discrete tokens. First, we fine-tune an Imagenet [19]-pretrained VQGAN on the SVG-Stack dataset with the VQ-adversarial reconstruction task. We find that using the features before the quantization yields better results. ConvNext [46] is a convolutional backbone, which we extract features before pooling. We start from a LAION-2B [75]-pretrained checkpoint. Finally, ViT CLIP [62] is based on the Visual Transformer (ViT) [22] and is well prepared for autoregressive tasks. We extract all output representations. We use a LAION-2B pre-trained model. During the training of StarVector, all the parameters of the image encoders are updated. We find that the best choice is using CLIP. We consider that the gains in performance come from CLIP using more visual tokens (257) than the other image encoders.

The adapter first projects the features from the original dimensionality D_v to a dimensionality $D_v \times 2$, followed by a Swish non-linear activation function and a linear projection to the LLM dimensionality D_l. Finally, we apply a layer normalization [5]. We initialize the adapter parameters with Glorot [27]. Dropout [79] of 0.1 is applied at the beginning. These hyperparameters were found using a random search on SVG-Fonts.

Our results show that image resolution is essential to capture fine-grained details like texts or high-frequency patterns. As seen in the SVG-Diagrams dataset in Figure 6), diagrams and figures are part of the SVG-Stack dataset and present challenging horizontal or vertical aspect ratios. When images have these aspect ratios, we make the image fit in the 224×224 resolution, losing much detail, especially for the OCR capabilities of reading rendered texts and accurately displaying them.

Additional results comparing image encoders can be found in Figures 19 and 22. These results show the boost in precision obtained when using CLIP. VGQAN and ConvNext often fail to capture the image's shape and the *path*'s trajectory. We note that ConvNext performs better than VQGAN. These differences are also due to the differences in the number of parameters. The CLIP ViT-L/14 model that we use consists of 290M parameters, VQGAN consists of 29M, and ConvNext consists of 179M parameters.

Generating SVGs from natural images is out of the scope of this project. However, future work should focus on adapting this approach to natural images, drawing from [50] and [12] to create a dataset of natural images and SVG pairs.

The selected image encoder architectures for StarVector include two variants: one with fewer parameters and reduced image resolution, based on the CLIP ViT-B/32 model, which processes images at 224x224 pixels and is utilized in StarVector-1B. The second variant, SigLip (`siglip-so400m302` patch14-384), has a larger number of parameters and processes images at a higher resolution of 384x384 pixels, and is employed in StarVector-8B. Given the positive results from the ViT architecture, we chose the SigLip variant due to its demonstrated effectiveness [97] and the enhanced resolution it provides.

11.2. Training

For training the StarVector model, we define the task of Image-to-SVG as an inverse rendering problem that converts a raster image (represented with visual tokens) into a

sequence of SVG code. This can be viewed as a sequence-to-sequence problem that models the translation between the image and SVG code domains. As detailed in Section 11, we utilize a CLIP ViT-B/32 for StarVector-1B and SigLip for StarVector-8B as image encoders, along with a non-linear adapter to generate a sequence of visual tokens.

The training process consists of two stages. In the first stage, the Image-to-SVG training phase, we construct sequences of visual tokens (produced by the image encoder and adapter) and SVG tokens, separated by a trigger token, `<svg-start>`. We train the LLM to learn these sequences on a large SVG-Stack dataset using a basic language modeling loss that calculates the cross-entropy loss in predicting the next token in a sequence based on the previous tokens.

This task enables the model to learn the concept of *drawing* with SVG vectors that resemble the input image. Importantly, this training can occur without supervision in the image domain (i.e., without pixel loss), relying solely on categorical cross-entropy loss for the LLM vocabulary introduced by the next-token prediction task.

In the second stage, we fine-tune the checkpoint from the first stage, which has learned SVG syntax through the Image-to-SVG task, on the Text-to-SVG task. During this phase, the image encoder is disregarded, as it becomes a Text-to-text task where the text instructions and SVG codes can be tokenized and processed directly by the LLM.

Training Details. We trained StarVector-1B on 1 node of 8 A100 80GB GPUs using Accelerate with DeepSpeed stage 2 and StarVector-8B on 8 nodes of 8 H100 80GB GPUs with Fully Shared Data Parallel (FSDP). For Image-to-SVG, we used total batch sizes of 128 and 512 for StarVector-1B and -8B, respectively, a learning rate of 1e-5, and the AdamW optimizer. To optimize memory and computation, we employed bf16 precision, FlashAttention2, and gradient checkpointing. StarVector-1B took 7 days to train, while StarVector-8B took 10 days, with both models completing 2 epochs.

We use HuggingFace Transformers [93] and PyTorch [58] for the implementation. We use a batch size of 2. Images are processed with a resolution of 224x224, as defined by the pre-trained CLIP image encoder, and process a maximum of 8192 tokens, considering the 257 visual tokens and the rest for the SVG tokens. We use gradient batch accumulation of 8 and train on a data parallel setup with 4 A100 80GB GPUs, having an effective batch size of 64. The learning rate is set to 5×10^{-4} for training, using AdamW optimizer [48] for approximately five days of training on the SVG-Stack dataset.

11.3. Generation

Here, we describe how to sample SVG code from our model. As a decoder-only LLM [42], StarVector first computes the key-value (KV) cache using the visual tokens from the image and then produces the initial set of output logits. This stage is often quick because the model can process the entire visual token sequence simultaneously [78]. The selected token from the output logits is then input back into the model, which generates logits for the subsequent token. This process is iteratively repeated until the model produces the desired quantity of tokens. Our approach uses architectural improvements for fast decoding, such as FlashAttention [18] and Multi-Query Attention [77]. We leverage vLLM to improve inference speed.

We perform a grid search on SVG-Emoji and SVG-Icons validation sets to select the correct sampling temperature. The choice of temperature does not strongly impact the results. However, a 1-point increase in performance is observed on CD for SVG-Emoji using temperatures close to 1.0.

We also present an ablation study of StarVector-1B popular decoding techniques [31, 52, 76, 85]. Specifically, we experiment with greedy decoding, beam search, and nucleus sampling with top-p. Results are shown in Table 7. The use of nucleus sampling with top-p=0.9 and temperature T=0.5 (no beam search) shows to be the best option. The beam search improves the greedy decoding baseline but does not work well when combined with nucleus sampling, increasing the inference time. In sum, we recommend nucleus sampling [31] with top p=0.9 and temperature between 0.5 and 0.9 for the best performance.

Are the SVGs valid and compilable? A common issue when generating SVGs with our approach is that the maximum token length of the LLM might not be sufficient to complete the SVG code, leading to compilation errors. We find that 85% of the generated SVG fit within the context length and compile successfully. The remaining incomplete samples are post-processed with `cairosvg` to produce a complete and compilable SVG. However, in some cases, parts of the image may be lost during this process. *With this technique, 100% of the generated SVGs are valid and compilable*.

Improving SVG Quality Through Sampling. The generation process is stochastic, meaning the outputs may sometimes take an incorrect *path*, leading to failed generations or repetitive patterns. To address this, we propose a simple baseline approach: generate k SVG outputs with varying sampling parameters (e.g., by adjusting the temperature),

then compare the outputs with the ground truth using a visual metric (we propose DinoScore) to select the most accurate result. In an ablation study conducted on SVG-Stack using StarVector-8B with $k = 1$ and $k = 5$, we observe a boost in DinoScore by 0.12. Empirically, after sampling 100 test samples, we find that 32% of the SVGs are more accurate when using $k = 5$, though this increases the generation time by a factor of k. The use of vLLM helps mitigate the slower sampling process, as it operates much faster. For further improvement in code generation, previous work has used MCTS techniques [7], which leverage visual feedback more effectively to guide the generation and enhance the stochastic sampling process.